

Exercise #4 - Real-Time Continuous Media

Mason McGaffin

Exercise #4

ECEN 5623 Real-Time Embedded Systems

University of Colorado Boulder

July 08, 2026

Contents

1 CAMERA VERIFICATION.....	3
2 CAMORAMA	5
3 CONTINUOUS VIDEO STREAM.....	9
SIMPLE-CAPTURE-1800.....	10
<i>PGM – 33ms sleep</i>	11
<i>PPM – 33ms sleep</i>	12
<i>PGM – No Sleep</i>	13
<i>PPM – No Sleep</i>	14
4 CONTINUOUS TRANSFORMATION	16
<i>Dump Frames - 33ms sleep</i>	17
<i>Dump Frames – No sleep</i>	18
<i>FFmpeg Pipe - No sleep</i>	19
5 FRAME TRANSFORMATION PERFORMANCE.....	21
A) ACQUISITION, TRANSFORMATION, AND WRITE-BACK PERFORMANCE	22
<i>Dump Frames</i>	23
<i>FFmpeg PIPE</i>	25
B) KEEPING ORIGINAL CAPTURE – WRITE BOTH.....	26
<i>Dump Frames</i>	27
<i>FFmpeg PIPE</i>	29
C) SEQUENCER.....	30
<i>Dump Frames</i>	31
<i>FFmpeg PIPE</i>	34
6 REFERENCES.....	38
7 APPENDIX	39

Note. For the RTES course, I, Mason McGaffin, will be using the Jetson Orin Nano as my embedded system of choice. Although this system is over resourced for our application, it is convenient to use given my current development situation.

1 Camera Verification

Obtain a Logitech C270 camera (or equivalent) and verify that it is detected by the Raspberry Pi USB driver:

a) Show that it has been detected by using `lsusb`, `lsmod` and `dmesg` kernel driver configuration tool to make sure your Logitech C270 USB camera is plugged in and recognized. Do `lsusb | grep -i logitech` and prove to me (and more importantly yourself) with that output (screenshot) that your camera is recognized.

b) Now, do `lsmod | grep video` and verify that the UVC driver is loaded as well (<http://www.ideasonboard.org/uvc/>).

c) To further verify, or debug if you don't see the UVC driver loaded in response to plugging in the USB camera, do `dmesg | grep video` or just `dmesg` and scroll through the log messages to see if your USB device was found.

d) Capture all output and annotate what you see with descriptions to the best of your understanding.

As seen in the output captured in Figure 1.1, the Logitech C920 webcam is recognized by the Jetson Orin Nano and the corresponding kernel drivers were also loaded. The first command, `lsusb`, shows the physical Logitech webcam device connection. The command shows the device is associated on USB Bus 1 as device 4 and is recognized by the Nano as the Logitech C920.

```
mason@jetson-orin-nano:~$ lsusb | grep -i logitech
Bus 001 Device 004: ID 046d:08e5 Logitech, Inc. HD Pro Webcam C920
mason@jetson-orin-nano:~$ lsmod | grep -i -e camera -e video
uvcvideo                106496  0
videobuf2_vmalloc       20480  1 uvcvideo
tegra_camera            217088  3 nvhost_isp5,nvhost_nvcsi_t194,nvhost_vi5
v4l2_dv_timings         36864  1 tegra_camera
nvhost_nvcsi            24576  1 tegra_camera
tegra_camera_platform   24576  4 nvhost_isp5,nvhost_nvcsi_t194,tegra_camera,nvhost_vi5
capture_ivc             28672  1 tegra_camera
tegra_camera_rtcpu      176128  2 capture_ivc,tegra_camera
ivc_bus                  24576  2 capture_ivc,tegra_camera_rtcpu
v4l2_fwnode              20480  1 tegra_camera
hsp_mailbox_client       20480  2 ivc_bus,tegra_camera_rtcpu
v4l2_async               20480  2 v4l2_fwnode,tegra_camera
videobuf2_dma_contig     24576  1 tegra_camera
videobuf2_memops         20480  2 videobuf2_vmalloc,videobuf2_dma_contig
videobuf2_v4l2           28672  2 tegra_camera,uvcvideo
videobuf2_common         61440  6 videobuf2_vmalloc,videobuf2_dma_contig,videobuf2_v4l2,tegra_camera,uvcvideo,videobuf2_memops
videodev                 249856  5 v4l2_async,videobuf2_v4l2,tegra_camera,uvcvideo,videobuf2_common
mc                        57344  5 videodev,videobuf2_v4l2,tegra_camera,uvcvideo,videobuf2_common
host1x_nvhost            40960  8 nvhost_isp5,nvhost_nvcsi_t194,nvidia,tegra_camera,nvhost_capture,nvhost_nvcsi,nvhost_vi5,nvidia_modeset
mc_utils                 16384  3 nvidia,nvgpu,tegra_camera_platform
```

Figure 1.1 Webcam device and kernel driver verification

The second command, `lsmod`, shows the kernel drivers associated with camera and video applications. These include the universal uvcvideo driver, modules for V4L2, and the native NVIDIA tegra modules for the Image Signal Processor hardware.

The third command, *dmesg*, shows the USB webcam was detected and initialized successfully, as seen in Figure 1.2. The V4L2 subsystem and NVIDIA tegra modules were loaded and the C920 webcam is identified and configured as a video input source.

```
mason@jetson-orin-nano:~$ sudo dmesg | grep -i -e cam -e video
[ 10.767712] videodev: Linux video capture interface: v2.00
[ 11.198078] tegra186-cam-rtcpu bc00000.rtcu: Adding to iommu group 27
[ 11.200700] tegra186-cam-rtcpu bc00000.rtcu: Trace buffer configured at IOVA=0xbff00000
[ 11.209098] tegra186-cam-rtcpu bc00000.rtcu: using cam RTCPU IRQ (219)
[ 11.209101] tegra186-cam-rtcpu bc00000.rtcu: tegra_camrtc_mon_create is successful
[ 11.209481] tegra186-cam-rtcpu bc00000.rtcu: firmware version cpu=rce cmd=6 sha1=e2238c99959d2df9350d393f04e1f44e5bef98cb
[ 11.982583] usb 1-2.2: Found UVC 1.00 device HD Pro Webcam C920 (046d:08e5)
[ 11.998721] input: HD Pro Webcam C920 as /devices/platform/bus@0/3610000.usb/usb1/1-2/1-2.2/1-2.2:1.0/input/input5
[ 11.998842] usbcore: registered new interface driver uvcvideo
[ 12.838177] tegra194-pcie 14160000.pcie: Phy link never came up
[ 13.839579] tegra194-pcie 14160000.pcie: Phy link never came up
[ 14.958162] tegra194-pcie 141e0000.pcie: Phy link never came up
[ 15.946223] tegra194-pcie 141e0000.pcie: Phy link never came up
```

Figure 1.2 Webcam initialization output

The following output in Figure 1.3 shows the devices and major/minor configuration of the webcam device. We see from the *v4l2-ctl --list-devices* command that the webcam is associated with video device 0 and 1 which are configured as character input devices ('c') in the video group with major 81.

```
mason@jetson-orin-nano:~$ v4l2-ctl --list-devices
NVIDIA Tegra Video Input Device (platform:tegra-camrtc-ca):
/dev/media0

HD Pro Webcam C920 (usb-3610000.usb-2.2):
/dev/video0
/dev/video1
/dev/media1

mason@jetson-orin-nano:~$ ls -l /dev/video*
crw-rw---- 1 root video 81, 0 Jun 26 00:24 /dev/video0
crw-rw---- 1 root video 81, 1 Jun 26 00:24 /dev/video1
mason@jetson-orin-nano:~$
```

Figure 1.3 V4L2 device configuration output

2 Camorama

If you do not have cheese or camorama, do “sudo apt-get install cheese” on your Raspberry Pi board [you may need to first do sudo apt-get update]. Camorama is supported on Ubuntu, but may not be on the R-Pi, so use cheese instead on the R-Pi. If for some reason cheese does not work for you, use an alternative application (e.g., install with “sudo apt-get install camorama” or one of these - <https://www.makeuseof.com/best-cameraapps-linux/>). This should not only install nice camera capture GUI tools, but also the V4L2 API (described well in this series of Linux articles - <http://lwn.net/Articles/203924/>). Provide a summary log of your installation.

a) Run camorama with no arguments and it should provide an interactive camera control session for your Logitech C2xx camera – provide a screen dump of the GUI.

b) If you have issues connecting to your camera do a “man camorama” and specify your camera device file entry point (e.g. /dev/video0). Run camorama and play with Hue, Color, Brightness, White Balance and Contrast, take an example image, and take a screen shot of the tool and provide both in your report. If you need to resolve issues with your network, USB, or sudo and your account, research this and please do so.

Camorama is not supported on modern linux systems. Cheese is a modern alternative, and it provides an interactive GUI camera control application, as seen in Figure 2.1.

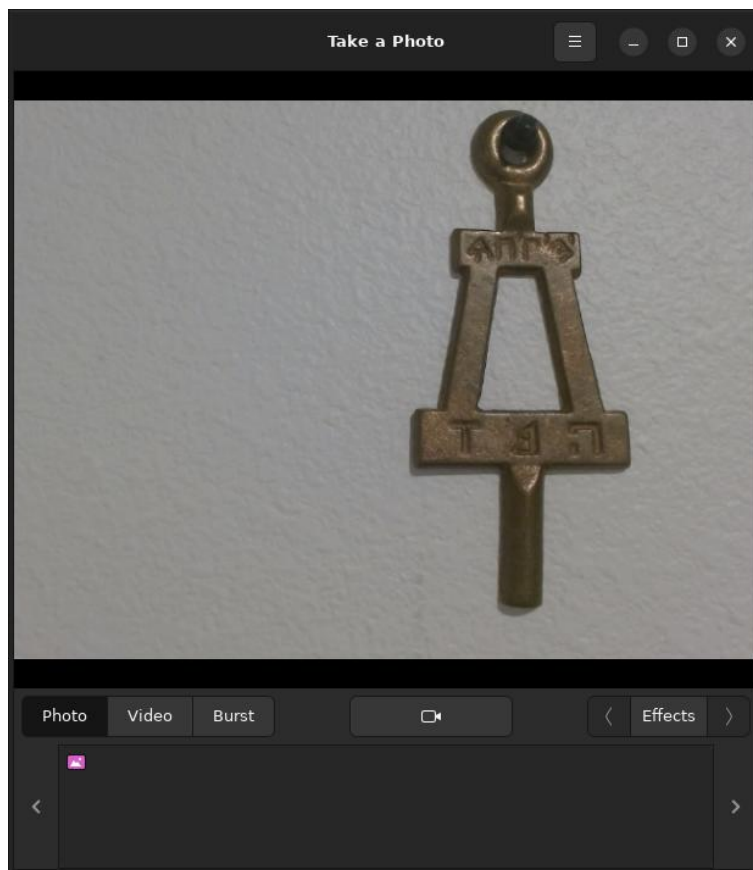


Figure 2.1 Cheese application GUI – no graphic effects

The cheese application has a control center accessed using the hamburger icon in the top right corner. The control center has three pages to modify the camera device and resolution, apply image effects, and modify capture configurations as shown in Figure 2.2.

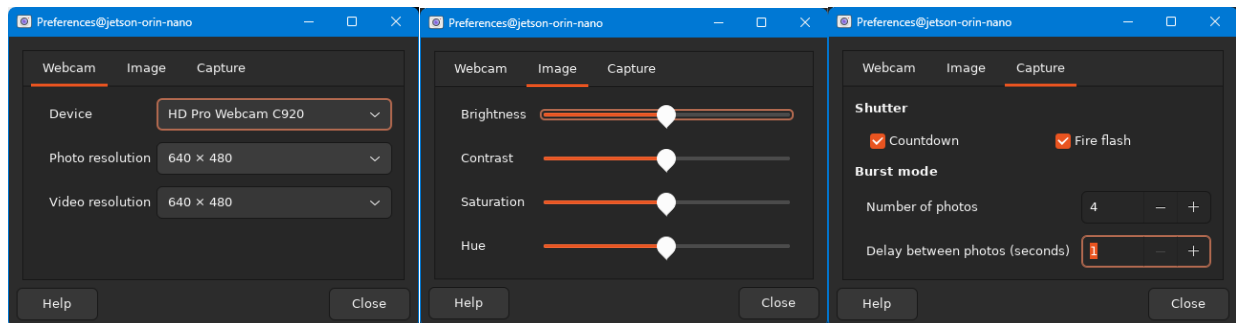


Figure 2.2 Cheese application preferences tabs

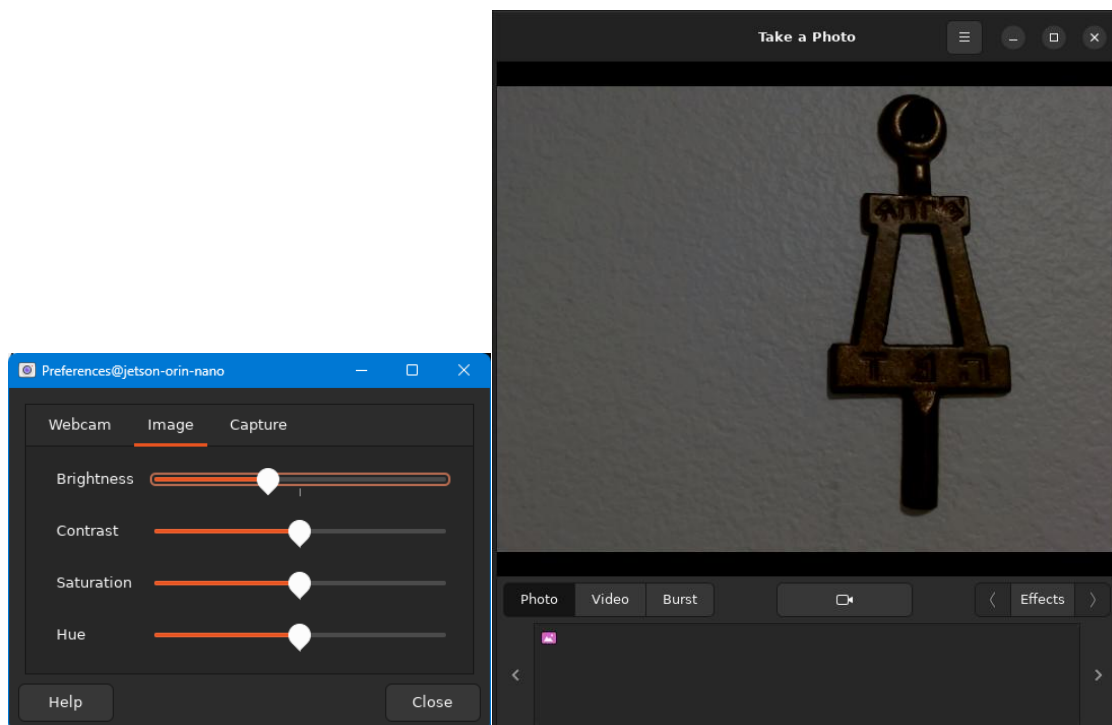


Figure 2.2 Lower brightness effect

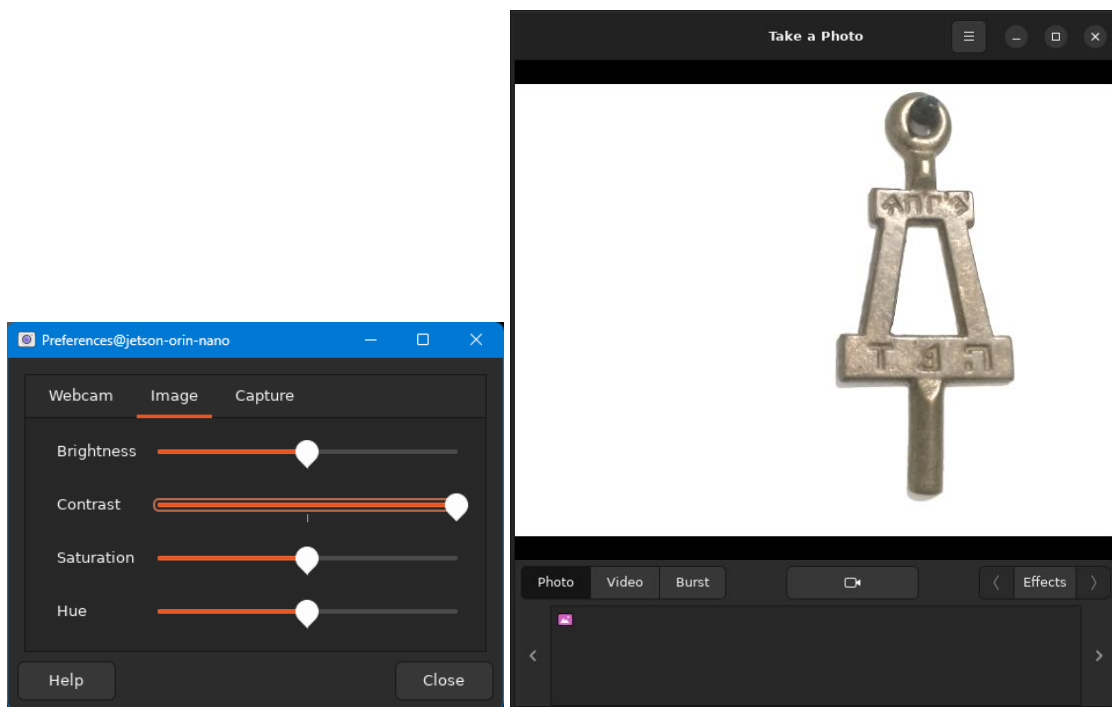


Figure 2.3 High contrast effect

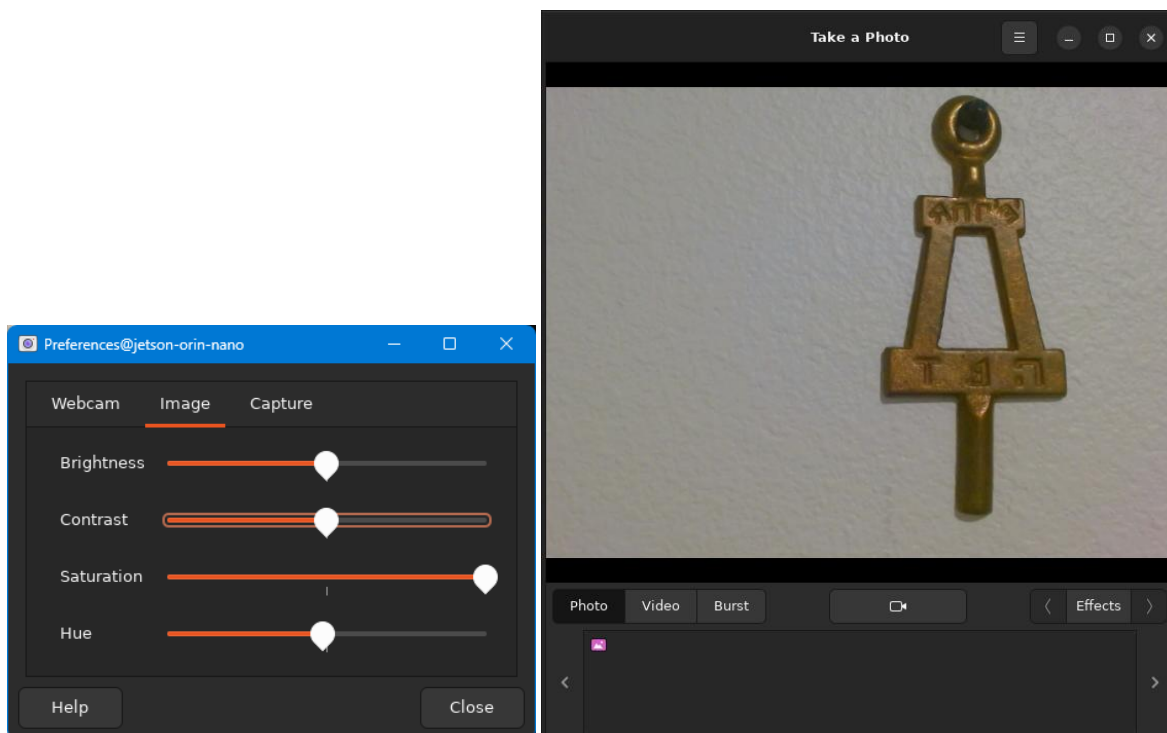


Figure 2.4 High saturation effect

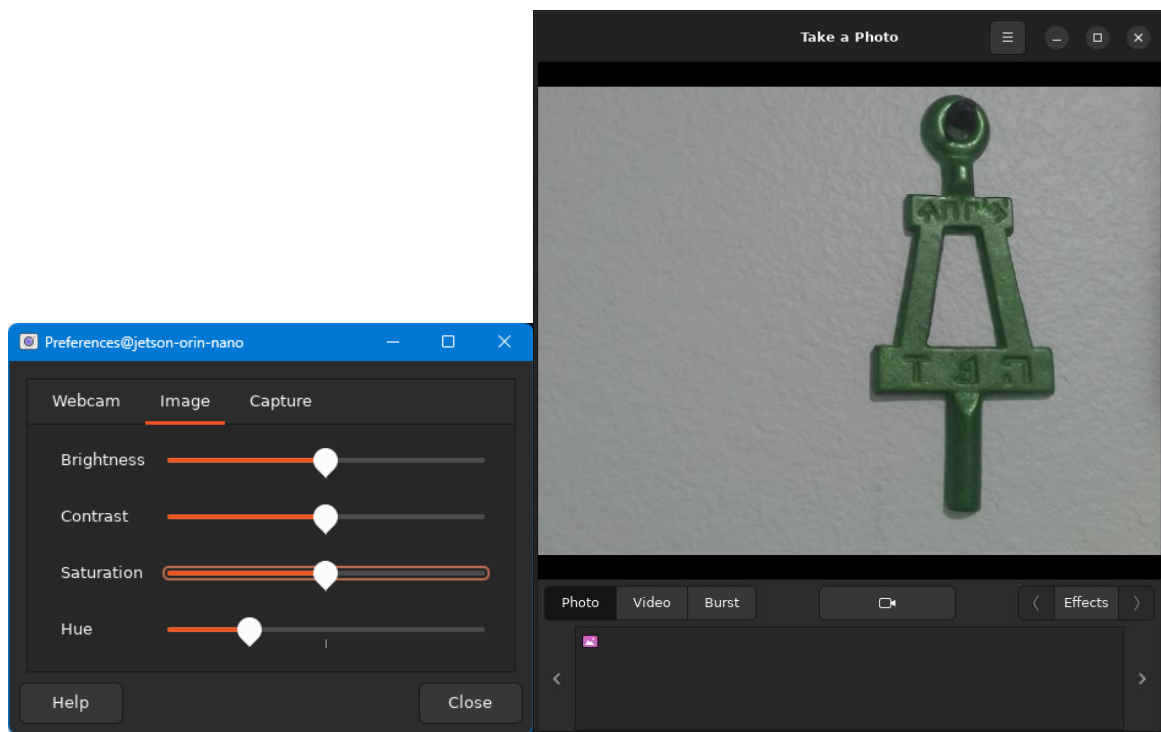


Figure 2.5 Green hue effect

3 Continuous Video Stream

Using your verified Logitech C270 camera on your Raspberry Pi, now verify that it can stream continuously.

a) Using starter code, capture a raw image buffer for transformation and processing using example code such as simple-capture. This basic example interfaces to the UVC and USB kernel driver modules through the V4L2 API. Provide a screen shot to prove that you got continuous capture to work with V4L2. To display images you have captured, you will find “eom” to be quite useful, which can be installed with “sudo apt-get install eom”.

Optional OpenCV with C++:

Alternatively, if you were to use OpenCV, which abstracts the camera interface and provides useful library functions for cameras and computer vision (computer_vision_cv4_tested). Note that simpler-capture-4 and other OpenCV examples will require installation of OpenCV 4.x on an R-Pi (if you have extra time to explore, here's some instructions to install OpenCV 4.x on the R-Pi – sudo apt update; sudo apt install libopencv-dev python3-opencv). The latest version of OpenCV, 4.x, has a C++ API and for camera frame capture, you can use and adapt the most up to date C++ code (simpler-capture-4). For ECEN 5623 it is recommended that you instead use V4L2 to avoid learning curves with C++ and the OpenCV API – use of real-time OpenCV and C++ is a topic covered in depth in the follow-on course ECEN 5763.

b) Download starter code (simple-capture-1800) that captures 1800 frames from your camera using V4L2 and run it as is first to test the code (use “make clean”, “make”, and run – note that make clean will remove test frames). Modify the code so that it uses syslog rather than printf to the console for tracing. Then, compare frame rate for PGM (graymap) and PPM (color) image write-back to the frame sub-directory on your flash filesystem and provide analysis of average and worst-case (lowest) frame rate seen for each.

In order to first get the [simple-capture](#) starter code to make without warnings, I had to change the strncat() calls to use snprintf(). The error “-Wstringoverflow” error which means that the operation of the strncat() call was unsafe. Transitioning to snprintf(), as seen in **this commit**, resolved this warning.

Despite the application building and running effectively, I noticed the application was not working all that great, due to the settings configured in the application. The application forces the resolution to be 320x240 and it is not very well processed.

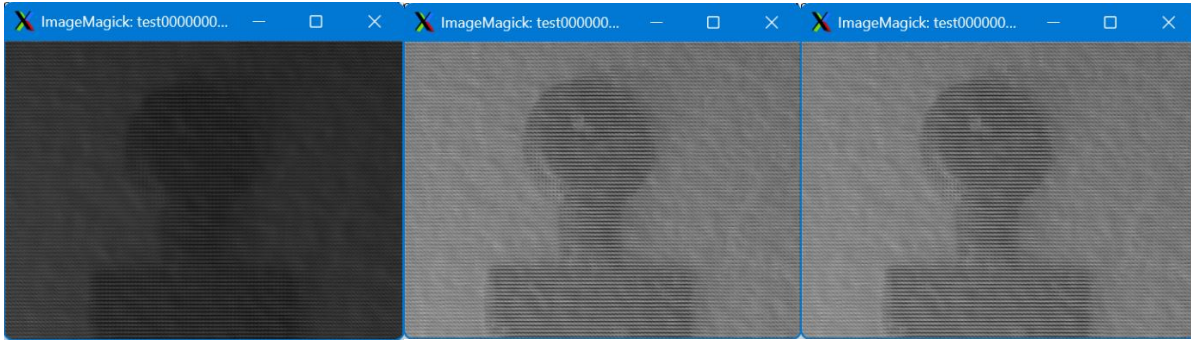


Figure 3.1 Frame 0 (left), 15 (middle), and 30 (right) from simple-capture application

simple-capture-1800

```
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaaffin/simple-capture-1800$ make
gcc -O0 -g -c capture.c
capture.c: In function 'dump_ppm':
capture.c:101:5: warning: 'strncat' specified bound 5 equals destination size [-Wstringop-overflow=]
  101 |     strncat(&ppm_dumpname[15], ".ppm", 5);
      |     ^~~~~~
capture.c:105:5: warning: 'strncat' specified bound 5 equals source length [-Wstringop-overflow=]
  105 |     strncat(&ppm_header[14], " sec ", 5);
      |     ^~~~~~
capture.c:107:5: warning: 'strncat' specified bound 19 equals source length [-Wstringop-overflow=]
  107 |     strncat(&ppm_header[29], " msec \n"HRES_STR" "VRES_STR"\n255\n", 19);
      |     ^~~~~~
capture.c: In function 'dump_pgm':
capture.c:133:5: warning: 'strncat' specified bound 5 equals destination size [-Wstringop-overflow=]
  133 |     strncat(&pgm_dumpname[15], ".pgm", 5);
      |     ^~~~~~
capture.c:137:5: warning: 'strncat' specified bound 5 equals source length [-Wstringop-overflow=]
  137 |     strncat(&pgm_header[14], " sec ", 5);
      |     ^~~~~~
capture.c:139:5: warning: 'strncat' specified bound 19 equals source length [-Wstringop-overflow=]
  139 |     strncat(&pgm_header[29], " msec \n"HRES_STR" "VRES_STR"\n255\n", 19);
      |     ^~~~~~
gcc -O0 -g -o capture capture.o -lrt
mason@jetson-orin-nano:~/RTES_5623/RTES-ECEE-5623-mcgaaffin/simple-capture-1800$ ./capture
FORCING FORMAT
allocated buffer 0
allocated buffer 1
allocated buffer 2
allocated buffer 3
allocated buffer 4
allocated buffer 5
frame -7: time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame -6: time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame -5: time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame -4: time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame -3: time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame -2: time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame -1: time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame 0: wrote 307200 bytes
Dump YUYV converted to YY size 614400
time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame 1: wrote 307200 bytes
Dump YUYV converted to YY size 614400
time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame 2: wrote 307200 bytes
Dump YUYV converted to YY size 614400
time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame 3: wrote 307200 bytes
Dump YUYV converted to YY size 614400
time_error.tv_sec=5, time_error.tv_nsec=281474487490400
frame 4: wrote 307200 bytes
```

Figure 3.2 simple-capture-1800 make and run terminal output before making fixes

In order to fix the warnings, I first made modifications to each of the `strncat()` commands to use `snprintf()` instead. I also changed the print statements to use `syslog` for tracing. I modified the `timer_error` usage to instead track read time, instantaneous and average frame rate .

Command for trace:

```
sudo tail -f /var/log/syslog |tee <test_name>.txt
```

The `plot_fps.py` script provides automated log parsing and visualization for the simple-capture-1800 syslog output. After each frame is captured and logged, the script uses a compiled regular expression (RegEx) to extract the frame number, elapsed time in seconds, instantaneous FPS, and running-average FPS from each matching log line. Once all matching lines are parsed, the script computes the minimum, maximum, and final average FPS across the session and prints a formatted summary table to the terminal.

The script then produces a single time-series plot showing instantaneous FPS (blue) and running-average FPS (red), plotted against elapsed time in seconds. The instantaneous FPS shows frame-to-frame variability caused by OS scheduling, V4L2 buffer availability, and filesystem write latency, while the average FPS line converges as more frames accumulate. The plot is saved as a PNG file alongside the input log.

PGM – 33ms sleep

```
mason@jetson-orin-nano:~/RTES_5623/ex4$ python3 plot_fps.py

Successfully parsed 1801 frames.
=====
                FPS PERFORMANCE METRICS
=====
Minimum FPS : 0.27
Maximum FPS : 28.40
Average FPS : 25.42
=====
Plot successfully generated and saved to 'pgm_1800_33ms_sleep_syslog_fps_performance.png'
```

Figure 3.3 simple-capture-1800 performance metrics – PGM output – 33ms sequencer sleep

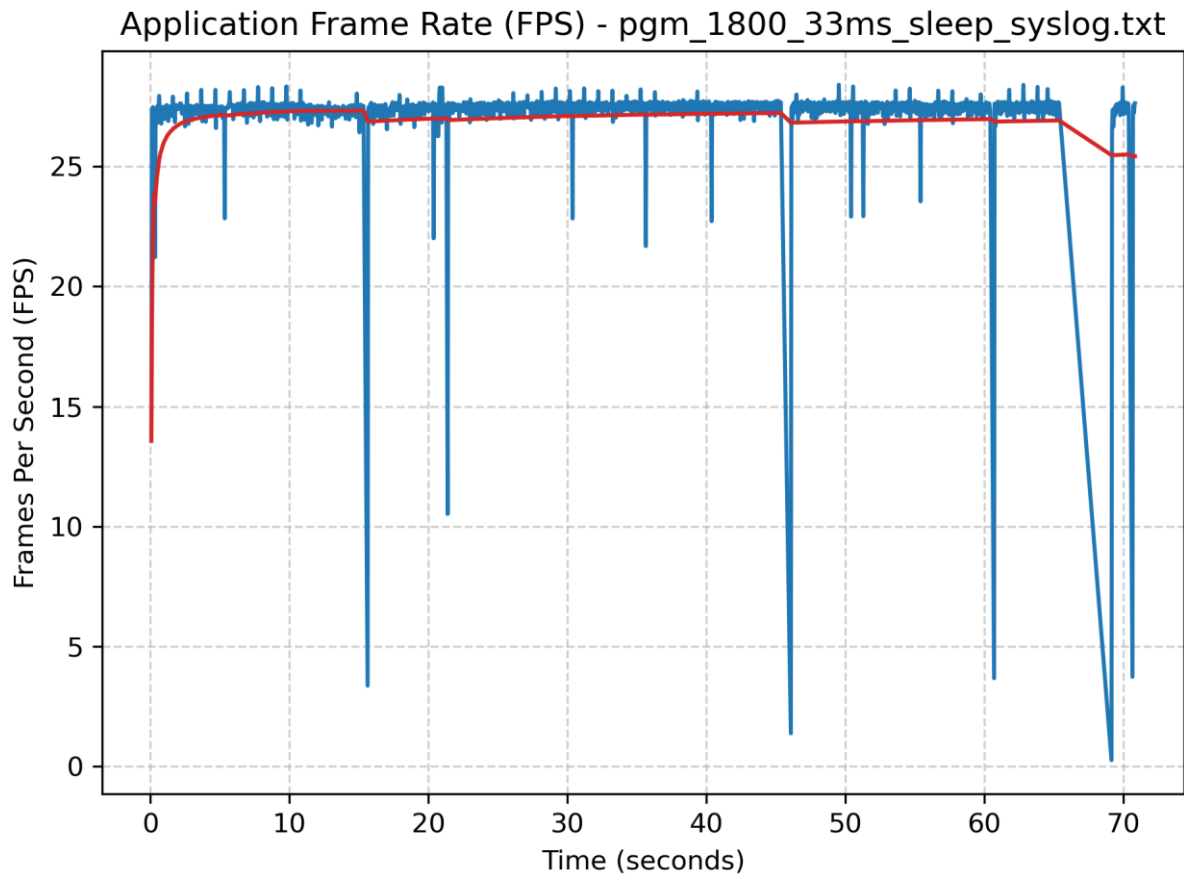


Figure 3.4 simple-capture-1800 frame rate chart – PGM output – 33ms sequencer sleep

PPM – 33ms sleep

```
mason@jetson-orin-nano:~/RTES_5623/ex4$ python3 plot_fps.py
Successfully parsed 1801 frames.
=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 0.21
Maximum FPS : 23.79
Average FPS : 16.93
=====
Plot successfully generated and saved to 'ppm_1800_33ms_sleep_syslog_fps_performance.png'
```

Figure 3.5 simple-capture-1800 performance metrics – PPM output – 33ms sequencer sleep

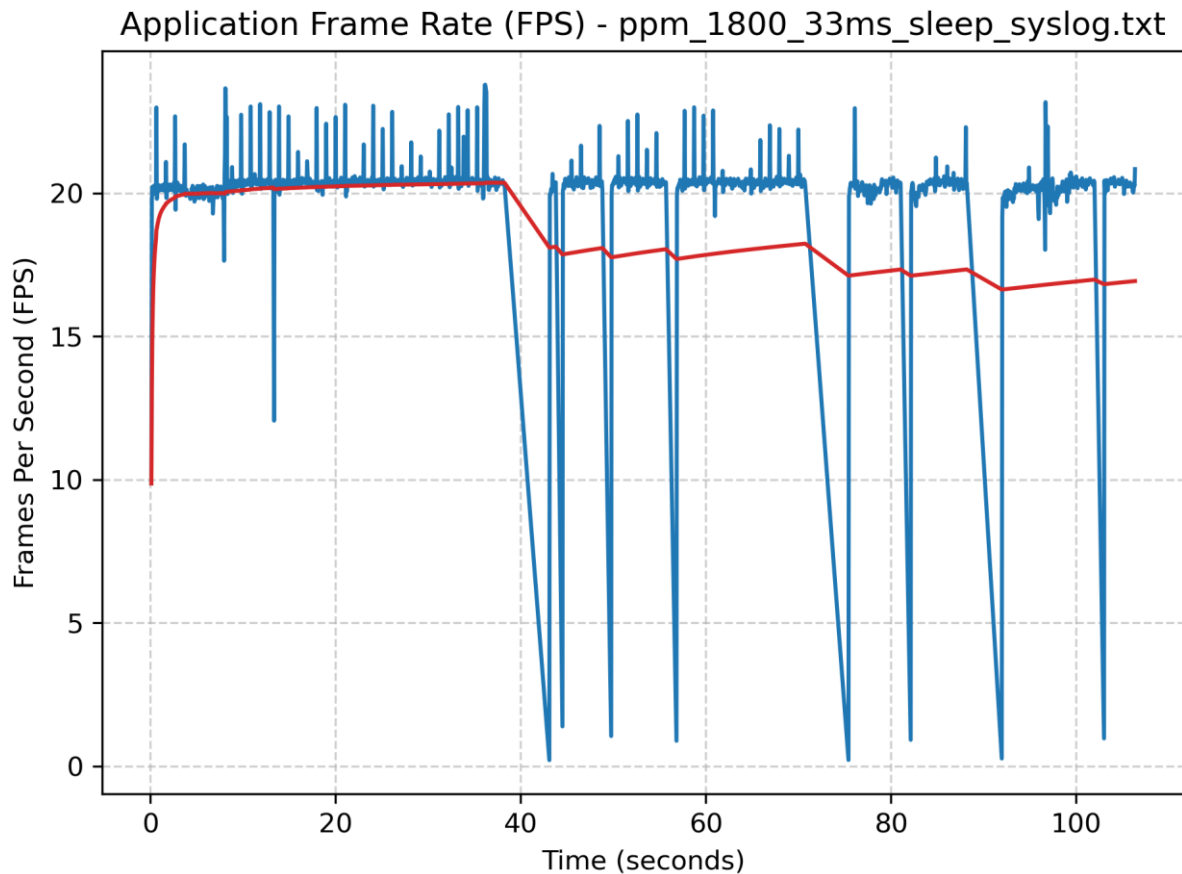


Figure 3.6 simple-capture-1800 frame rate chart – PPM output – 33ms sequencer sleep

As observed above, both PGM and PPM outputs with the 33ms sleep experience write-back delays. This is most likely due to buffer handling or kernel preemption. These delays are shown by the instantaneous frame rate curves. Clearly, the PPM output has higher quantity and length of these delays, most likely because it is writing triple the amount of data.

PGM – No Sleep

```
mason@jetson-orin-nano:~/RTES_5623/ex4$ python3 plot_fps.py

Successfully parsed 1801 frames.
=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 1.08
Maximum FPS : 401.88
Average FPS : 29.34
=====
Plot successfully generated and saved to 'pgm_1800_no_sleep_syslog_fps_performance.png'
```

Figure 3.7 simple-capture-1800 performance metrics – PGM output – no sleep

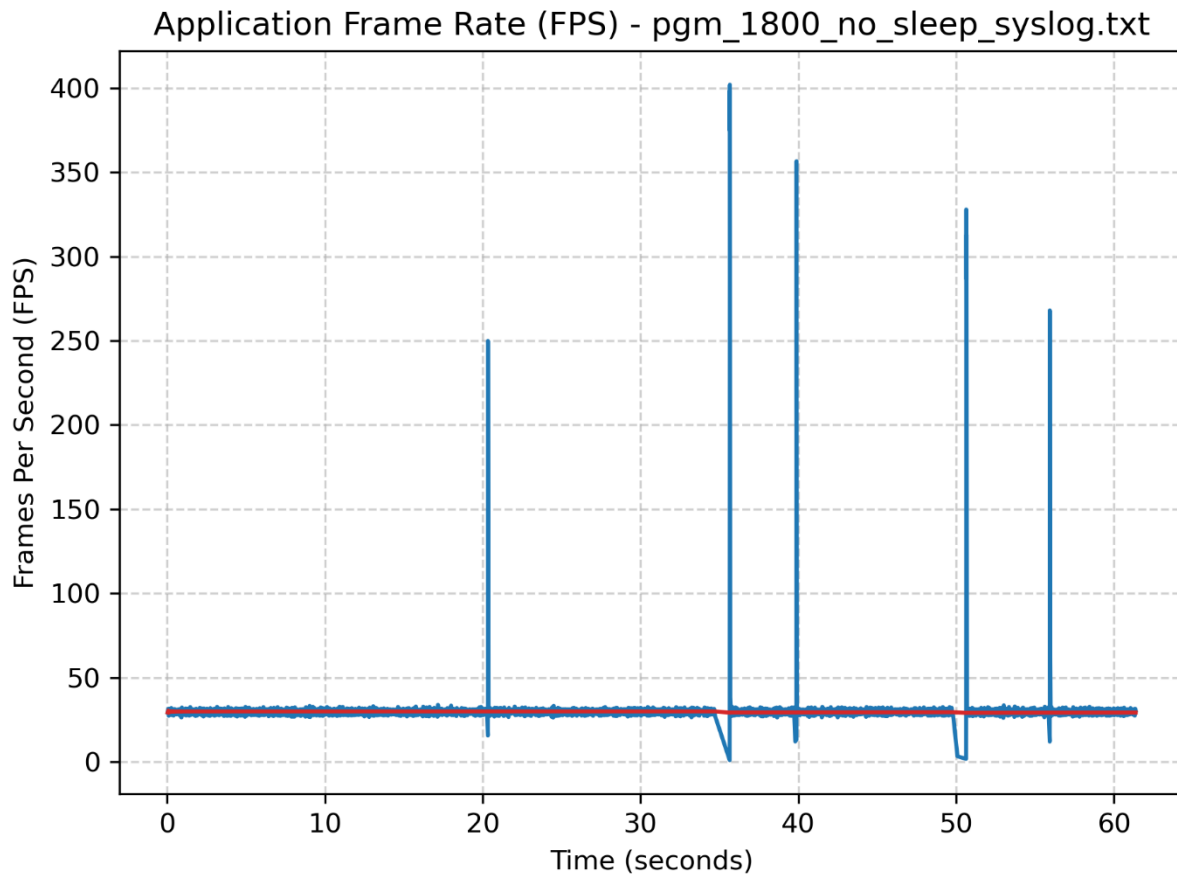


Figure 3.8 simple-capture-1800 frame rate chart – PGM output – no sleep

PPM – No Sleep

```
mason@jetson-orin-nano:~/RTES_5623/ex4/q3 (RTES_5623::ex4)
$ python3 plot_fps.py

Successfully parsed 1801 frames.

=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 0.35
Maximum FPS : 23.58
Average FPS : 18.31
=====

Plot successfully generated and saved to 'ppm_1800_no_sleep_syslog_fps_performance.png'
mason@jetson-orin-nano:~/RTES_5623/ex4/q3 (RTES_5623::ex4)
```

Figure 3.9 simple-capture-1800 performance metrics – PPM output – no sleep

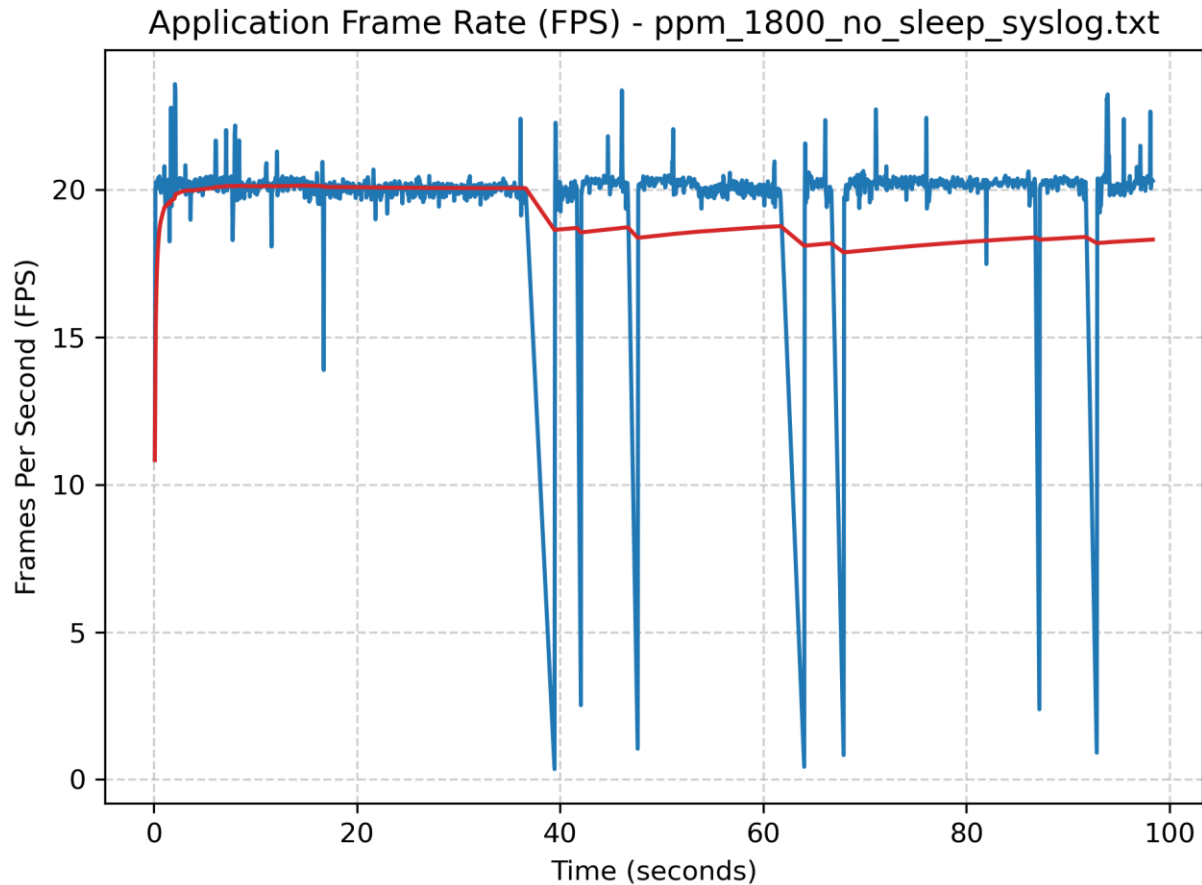


Figure 3.10 simple-capture-1800 frame rate chart – PPM output – no sleep

By removing the sleep call, the application is able to run as fast as possible, which exposes the I/O bottleneck. PPM still achieves a relatively low frame rate (~18FPS) while PGM output nearly reaches the desired 30FPS. The Instantaneous FPS for PGM output shows less variability, except for the one or two random spikes. I am unsure why those happened. But, in the PPM, we still see the write-back delays that cause a decrease in average FPS. This exercise shows how sleep based limiters can mask I/O ceilings at the cost of throughput. Neither of these approaches is ideal for real-time embedded systems as they do not meet the expected deadlines defined by the 30FPS hardware camera.

4 Continuous Transformation

Choose ONE continuous transformation for each acquired frame such as color conversion (Lecture-Cont-Media.pdf, slide 5), conversion to grayscale (slide 6), negative image ($\text{new_pixel} = \text{saturation} - \text{pixel}$), brightness/contrast adjustment (e.g., /cbrighten/brighten.c), or sharpening (/sharpen-psf/sharpen.c) and consider how to apply this to a real-time stream of image frames acquired from your camera rather than just one image file (PPM in the examples).

Optional: If you want to use OpenCV (not required, but can be fun and interesting if you have extra time) look at examples from computer_vision_cv3_tested/ then you can choose transformations such as the diff-interactive/, simple-canny-interactive/, simplehoughinteractive/, etc. Note that these examples were written for OpenCV 3.x, not OpenCV 4.x. Newer versions of this code can be found in computer_vision_cv4_tested including diffinteractive-updated. This is not required, but noted for students interested in real-time computer vision, which is covered in depth in ECEN 5763, Embedded Machine Vision and Intelligent Automation.

- a) Show a screen shot to prove you built and ran the code and a before and after transform example image.*
- b) Provide a detailed explanation of your code and research uses for the continuous transformations and describe how this might be used in a machine vision application.*
- c) What was the best frame rate for continuous transform processing for the transform you chose to investigate?*

Note that your real bottleneck may be writing your frames to your flash file system device, so try disabling frame write-back to the frame sub-directory and transforming frames, but not saving them.

The brightening transform implemented in continuous_brighten.c applies a linear gain and offset to each pixel after YUYV-to-RGB conversion. For each pixel, the channel value is scaled by alpha (1.35) and shifted by beta (45), then clamped to the [0, 255] range. This is a per-pixel point operation with $O(N)$ complexity and no spatial neighborhood access, making it among the fastest possible image transforms. The integer YUV-to-RGB conversion in yuv2rgb_brighten() uses fixed-point arithmetic to avoid floating-point overhead during the color space conversion, and the brightening step is folded directly into the conversion before clamping.

Brightness adjustment is a foundational preprocessing step in real-time machine vision systems. For example, it is used in industrial inspection to create consistent and uniform illumination for reliable feature detection. Automotive ADAS cameras apply similar techniques to normalize exposure between weather conditions, ensuring that lane markings and pedestrians are detected. Medical imaging pipelines use brightness normalization to compensate for inter-session variation in scanner gain or patient positioning. In all of these applications, the low computational cost of a

per-pixel linear transform means it can be applied at full frame without compression.

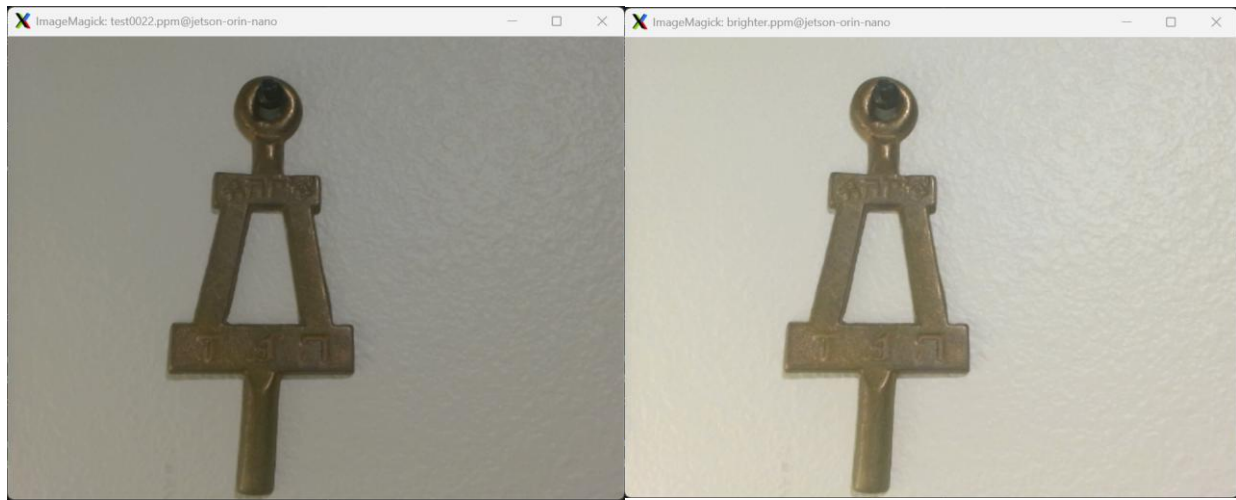


Figure 4.1 Before and after transform example image

Dump Frames - 33ms sleep

```
mason@jetson-orin-nano:~/RTES_5623/ex4$ python3 plot_fps.py  
  
Successfully parsed 300 frames.  
=====
```

FPS PERFORMANCE METRICS
Minimum FPS : 9.58
Maximum FPS : 22.94
Average FPS : 19.17

```
=====
```

Plot successfully generated and saved to 'continuous_brighten_syslog_fps_performance.png'

Figure 4.2 continuous_brighten performance metrics – DUMP_FRAMES – 33ms sequencer sleep

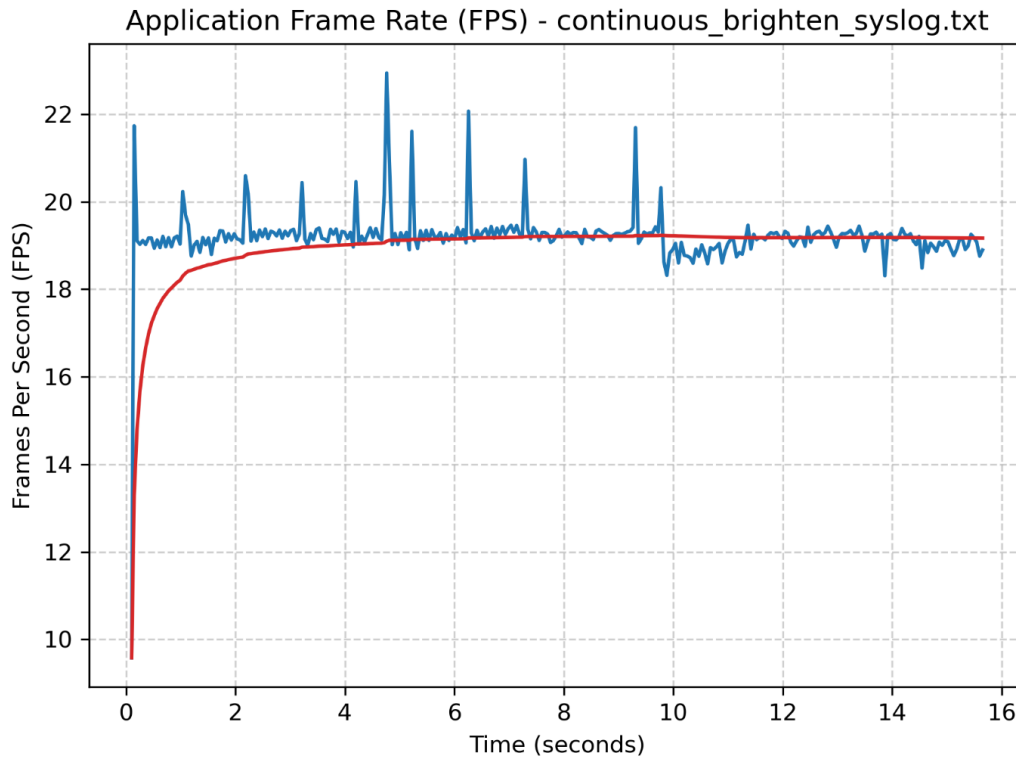


Figure 4.3 continuous_brighten frame rate chart – DUMP_FRAMES – 33ms sequencer sleep

```
FFmpeg -framerate 19.17 -i frames/test%04d.ppm -c:v libx264 -pix_fmt yuv420p
continuous_brighten_33ms.mp4
```

Command 4.1 FFmpeg command to create MP4 video from frames

Dump Frames – No sleep

```
mason@jetson-orin-nano:~/RTES_5623/ex4$ python3 plot_fps.py

Successfully parsed 300 frames.
=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 6.51
Maximum FPS : 101.18
Average FPS : 29.87
=====
Plot successfully generated and saved to 'continuous_brighten_syslog_no_sleep_fps_performance.png'
```

Figure 4.4 continuous_brighten performance metrics – DUMP_FRAMES – no sleep

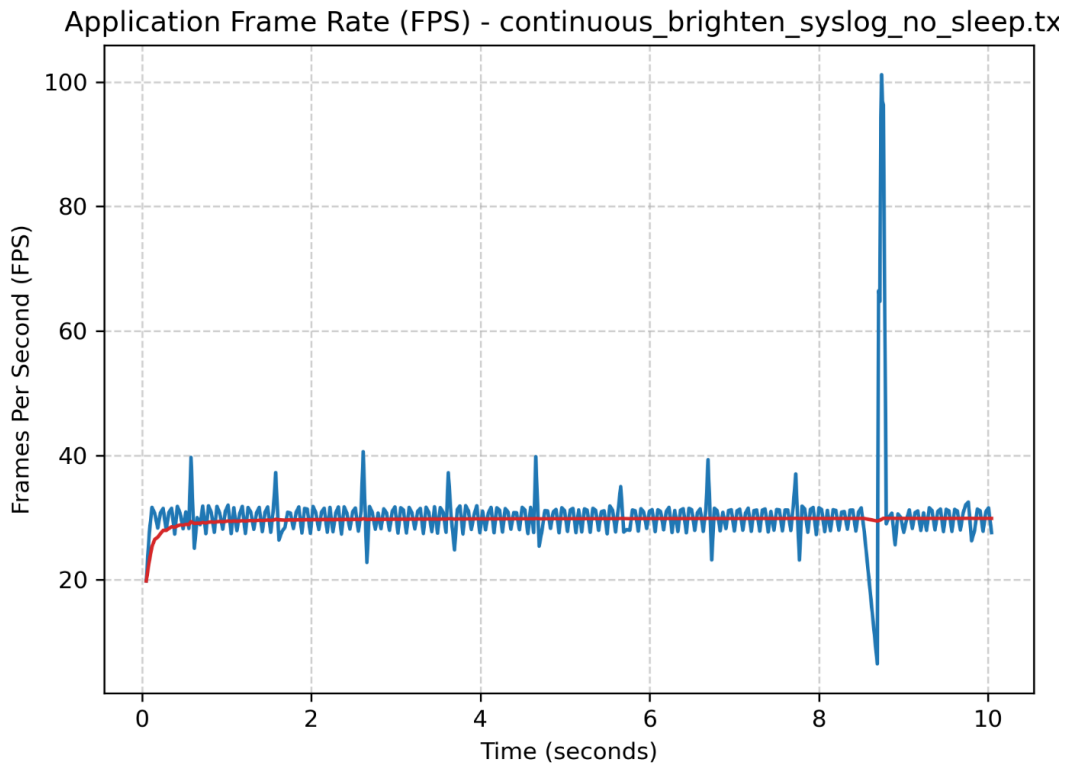


Figure 4.5 continuous_brighten frame rate chart – DUMP_FRAMES – no sleep

```
FFmpeg -framerate 29.87 -i frames/test%04d.ppm -c:v libx264 -pix_fmt yuv420p
continuous_brighten_no_sleep.mp4
```

Command 4.2 FFmpeg command to create MP4 video from frames

FFmpeg Pipe - No sleep

```
mason@jetson-orin-nano:~/RTES_5623/ex4$ python3 plot_fps.py

Successfully parsed 300 frames.
=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 10.80
Maximum FPS : 58.97
Average FPS : 29.89
=====
Plot successfully generated and saved to 'piped_continuous_brighten_syslog_fps_performance.png'
```

Figure 4.6 continuous_brighten performance metrics – FFmpeg pipe – no sleep

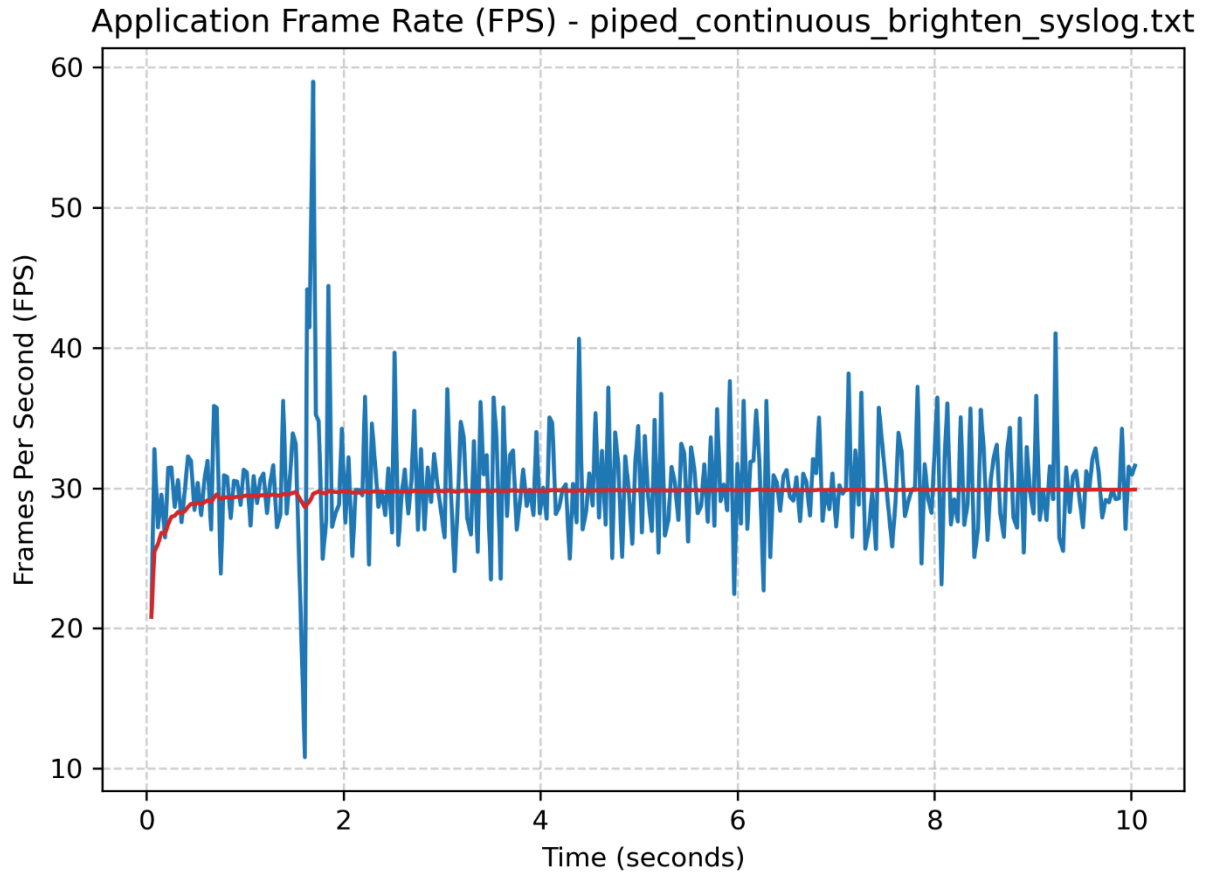


Figure 4.7 continuous_brighten frame rate chart – FFmpeg pipe – no sleep

The above figures shows that the transform itself is negligible. The sequencer sleep limits the throughput but provides more a stable instantaneous FPS. Without the sleep, the DUMP_FRAMES mode is much faster (~29 FPS). As mentioned in the instructions, writing to disk can also be a bottleneck, especially for larger PPM files. In order to combat this, I introduced an option, controlled by the DUMP_FRAMES macro (not enabled), that directs the frame data to a FFmpeg pipe instead of writing to disk. This option creates a MPEG4 video directly as opposed to writing out each frame and having to manually call the process. This transfers the latency from I/O operations to FFmpeg pipe encoding and buffering operations. The result is a higher frame rate with the ability to support higher FPS hardware.

5 Frame Transformation Performance

Using a Logitech C270 (or equivalent), choose ONE real-time frame transformation and measure the frame processing time (WCET). I have made some videos on the “L-N9.#-Final-Project” (found in Lightboard-Studio-rough-cuts) that demonstrate the analysis of a single thread design (simple-capture-1800) and use of syslog for tracing (CW21-C4-simplecapture-1800-starter-code-for-project) and you may also find simple-capture1800-syslog/ useful.

a) Please implement and compare transform performance in terms of average transform processing time and overall average frame rate for acquisition, transformation and writeback of only the transformed frame. Note the average for each step clearly (acquisition, processing, write-back).

b) Please implement and compare transform performance in terms of average frame rate for transformation and write-back of both the transformed frame and captured.

c) Based on average analysis for transform frame only write-back, pick a reasonable soft real-time deadline (e.g., if average frame rate is 10 Hz, choose a deadline of 100 milliseconds) and convert the processing to SCHED_FIFO and determine if you can meet deadlines with predictability. Note, here are some examples of monotonic service analysis and jitter (here). The drift is shown as a red polynomial trend line and jitter is a plot of raw delta-t data compared to expected.

The sharpening transform implemented in `cont_sharpen.c` applies a 3x3 Point Spread Function (PSF) convolution kernel to each non-border pixel of the RGB frame. The PSF has a center weight $K+1$ ($K=4.0$, center=5.0) and surrounding weights of $-K/8$ (surrounding=-0.5) each. For every pixel, 9 operations are performed for each of the three color channels, totaling 27 floating-point operations per pixel. For a 640x480 frame, this yields approximately 8.3 million floating-point operations per frame, excluding border pixels. The result is clamped and written to a separate output buffer, preserving the original for optional dual write-back.

The sharpening transform is chosen for Section 5 analysis rather than brightening specifically because it has substantial and measurable execution time, on the order of tens of milliseconds on the Jetson Orin Nano, representative of real computer vision operations. The brighten transform executes in under a millisecond and provides no meaningful WCET data for real-time timing analysis. Understanding the sharpen transform WCET, jitter, and deadline behavior under a SCHED_FIFO sequencer is therefore more instructive for real-time computer vision design.

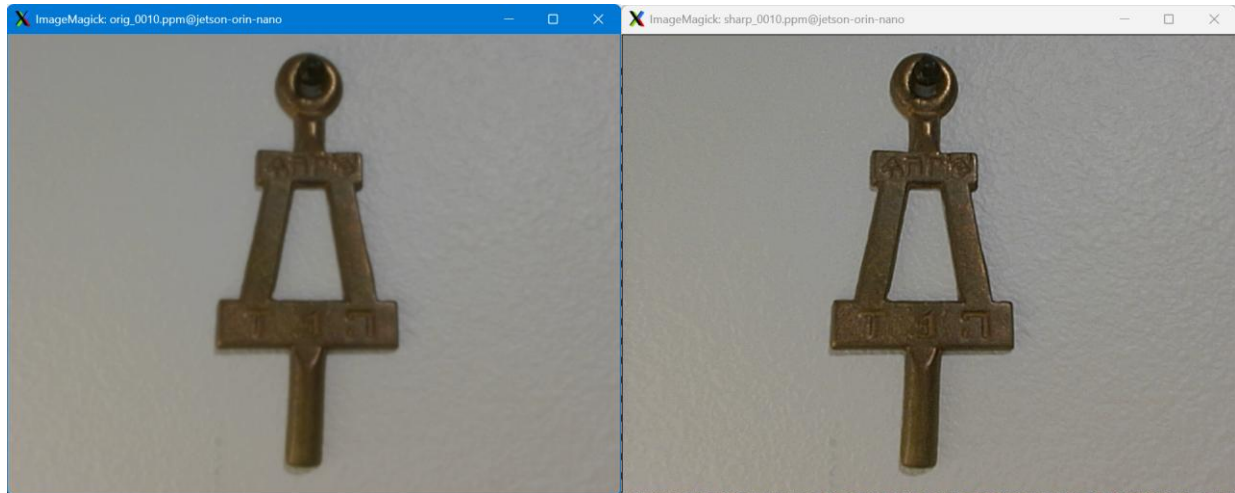


Figure 5.1 Sharpen transform (right) on example capture (left)

a) Acquisition, Transformation, and Write-Back Performance

The frame pipeline in `cont_sharpen.c` is decomposed into three independently timed stages, each bracketed with `CLOCK_MONOTONIC` timestamps to produce per-frame latency measurements logged to `syslog`. First, acquisition calls `VIDIOC_DQBUF` to dequeue a completed DMA buffer from the V4L2 kernel driver; this step is gated by the camera hardware frame rate and is typically fast, but may stall if the camera has not yet produced a new frame. Second, the transform stage performs the YUYV-to-RGB color space conversion followed by the 3x3 PSF convolution sharpening, writing the sharpened output to a separate buffer; this is the most compute-intensive stage. Third, write-back either calls `dump_ppm()` to create individual PPM files on the flash filesystem, or pipe raw RGB data into FFmpeg for encoding. The write-back stage is often a dominant bottleneck: flash filesystem operations involve repeated `open()`, `write()`, and `close()` syscalls with latency due to flash wear-leveling and block allocation, while pipe write-back depends on FFmpeg encoding, throughput, and buffering.

The `plot_fps.py` script was adapted to retain core FPS parsing as well as parse additional patterns for per-stage timing data that are logged from `cont_sharpen.c`. These patterns populate three new lists and are used to calculate and print min/max/average duration for each processing step (acquisition, transformation, and write-back). The script retains the function to plot the average and instantaneous FPS. It also allows new functionality to plot the instantaneous process time (solid lines) and overall average processing time (dashed lines) for each of the steps. The script also organizes both output PNG charts in an output directory (`perf_<logname>/`)

Dump Frames

```
mason@jetson-orin-nano:~/RTES_5623/ex4/q5 (RTES_5623::ex4)
$ python3 plot_perf.py

Successfully parsed 300 frames.

=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 6.4763
Maximum FPS : 13.2197
Average FPS : 12.9801
=====

ACQUISITION DURATION (ms)
=====
Minimum : 0.0068
Maximum : 0.0595
Average : 0.0092
=====

TRANSFORM DURATION (ms)
=====
Minimum : 73.6843
Maximum : 75.9065
Average : 74.8343
=====

WRITE-BACK DURATION (ms)
=====
Minimum : 1.5242
Maximum : 2.3483
Average : 1.8013
=====

FPS plot successfully generated and saved to 'perf_cont_sharp_dump/fps_performance.png'
Processing time plot successfully generated and saved to 'perf_cont_sharp_dump/step_performance.png'
```

Figure 5.2 cont_sharpen frame rate and execution time metrics – DUMP_FRAMES

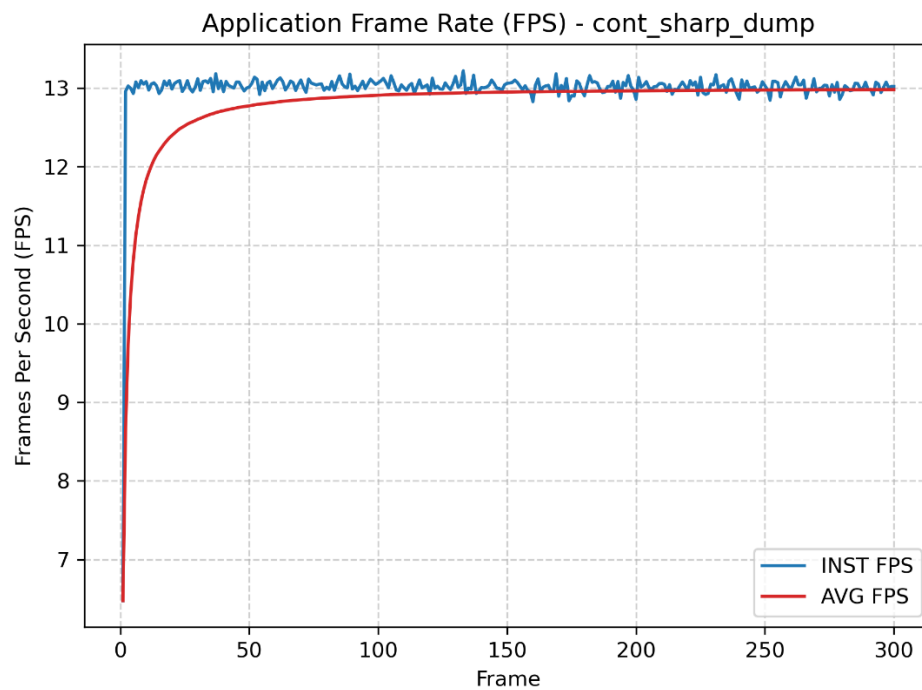


Figure 5.3 cont_sharpen frame rate chart – DUMP_FRAMES

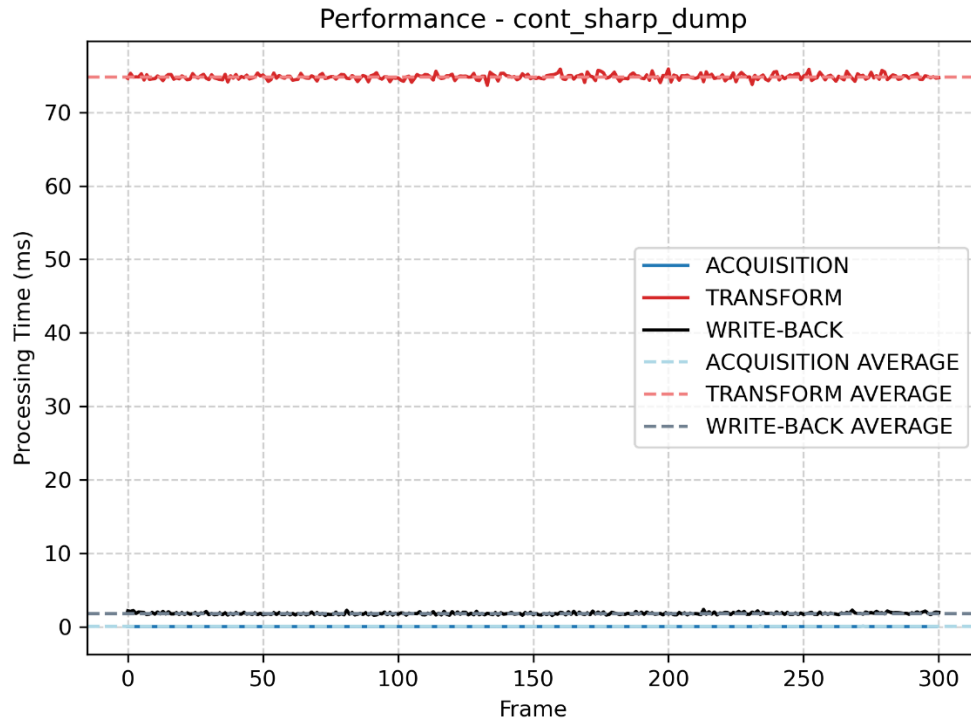


Figure 5.4 cont_sharpen execution time chart – DUMP_FRAMES

In DUMP_FRAMES mode, acquisition time is consistently low as it is paced by the camera hardware producing frames at approximately 30 fps. The transform time reflects the cost of the 3x3 PSF convolution over the 640x480 frame and is relatively stable across frames, as it is a deterministic compute workload with no I/O dependency. The write-back step shows the highest variability: each frame requires creating, writing, and closing a PPM file on the flash filesystem, and flash storage latency can spike unpredictably due to garbage collection or wear-leveling events. The overall instantaneous FPS is therefore dominated by write-back jitter rather than transform time. The average FPS converges to a value well below the camera hardware frame rate, reflecting the combined pipeline latency including transform and flash I/O overhead.

FFmpeg PIPE

```
mason@jetson-orin-nano:~/RTES_5623/ex4/q5 (RTES_5623::ex4)
$ python3 plot_perf.py

Successfully parsed 300 frames.

=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 6.4745
Maximum FPS : 13.5373
Average FPS : 13.2534
=====
ACQUISITION DURATION (ms)
=====
Minimum : 0.0065
Maximum : 0.0247
Average : 0.0099
=====
TRANSFORM DURATION (ms)
=====
Minimum : 72.5997
Maximum : 83.3983
Average : 73.6022
=====
WRITE-BACK DURATION (ms)
=====
Minimum : 1.0002
Maximum : 5.6038
Average : 1.3452
=====
FPS plot successfully generated and saved to 'perf_cont_sharp_pipe/fps_performance.png'
Processing time plot successfully generated and saved to 'perf_cont_sharp_pipe/step_performance.png'
```

Figure 5.5 cont_sharpen frame rate and execution time metrics – FFmpeg pipe

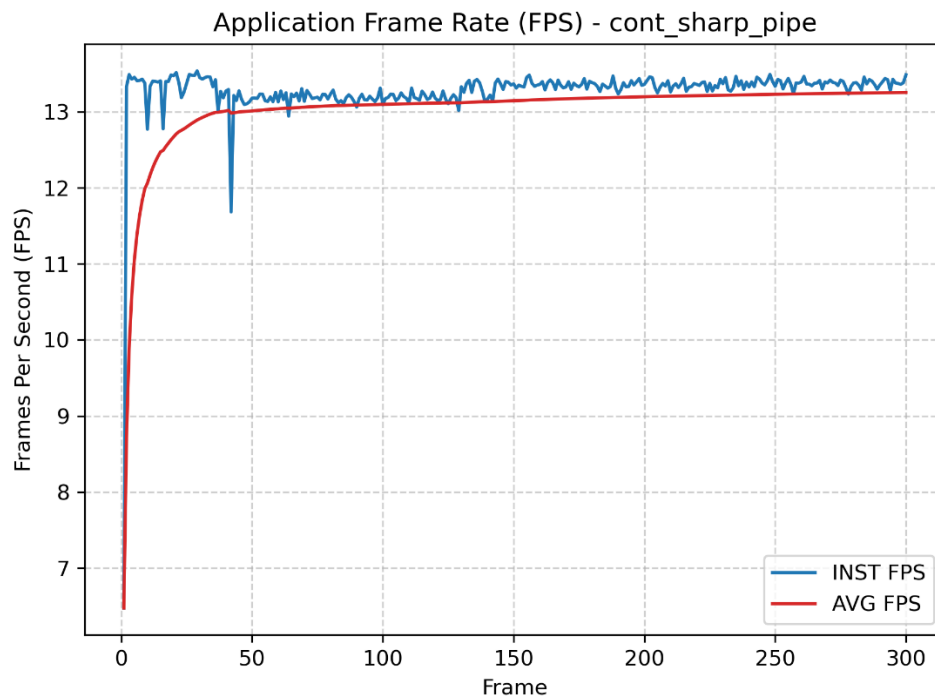


Figure 5.6 cont_sharpen frame rate chart – FFmpeg pipe

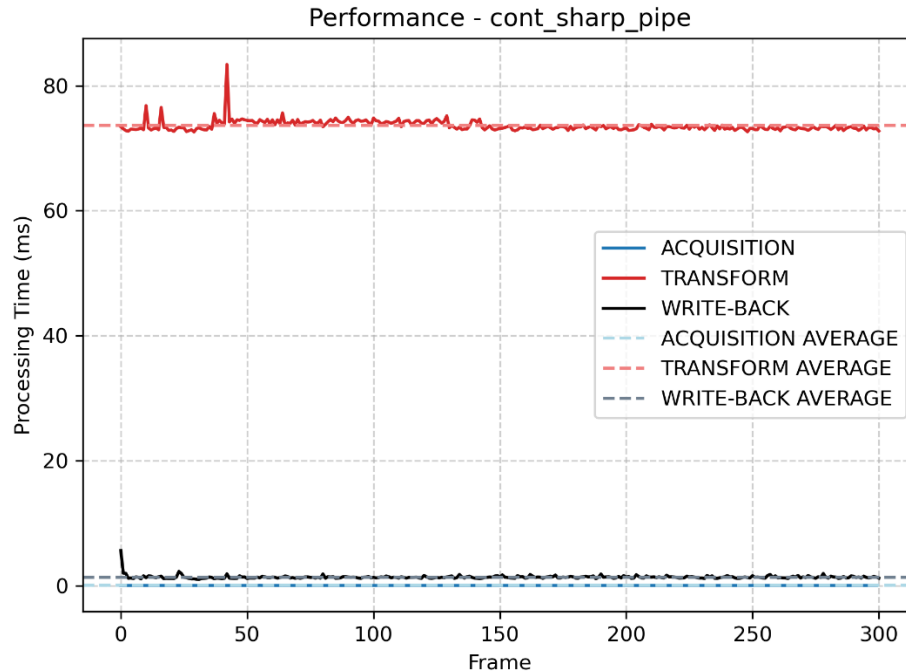


Figure 5.7 cont_sharpen execution time chart – FFmpeg pipe

In pipe mode, the write-back stage replaces per-frame file creation with a single call into FFmpeg pipe. This eliminates the repeated `open()` and `close()` overhead of DUMP_FRAMES mode and generally reduces write-back latency and variability. The average FPS in pipe mode is typically higher than in DUMP_FRAMES mode because the FFmpeg process buffers input and disperses encoding overhead across multiple frames. However, if FFmpeg cannot keep pace with incoming data, it will block until the pipe buffer drains, introducing occasional write-back spikes. Compared to DUMP_FRAMES mode, pipe mode offers a more consistent write-back latency profile, resulting in smoother instantaneous FPS readings and a higher average FPS.

b) Keeping Original Capture – Write Both

The `WRITE_BOTH` preprocessor macro in `cont_sharpen.c` controls whether both the original captured frame and the sharpened output are written per cycle. When defined, `process_image()` writes two outputs: in DUMP_FRAMES mode, it calls `dump_ppm()` twice per frame, once with prefix "sharp_" and once with prefix "orig_", saving both outputs to the frames/ directory. In pipe mode, two separate FFmpeg processes are opened (`ffmpeg_pipe` for the sharpened stream and `ffmpeg_pipe2` for the original), and a separate call is made to each pipe per frame.

The transform computation itself is unaffected by `WRITE_BOTH`, since the sharpen operation is only computed once regardless. The impact is entirely in the write-back stage: in DUMP_FRAMES mode, the write-back time approximately doubles because two files of equal size are written per frame. In pipe mode, two sequential calls are made, and the write-back time is the sum of both, which may exceed twice the single-pipe write time. The result is a reduction

in average FPS compared to the single-output configuration in part a, due to the increased I/O with two pipes.

Dump Frames

```
mason@jetson-orin-nano:~/RTES_5623/ex4/q5 (RTES_5623::ex4)
$ python3 plot_perf.py

Successfully parsed 300 frames.

=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 1.2040
Maximum FPS : 13.7878
Average FPS : 12.9562
=====
ACQUISITION DURATION (ms)
=====
Minimum : 0.0056
Maximum : 0.0198
Average : 0.0080
=====
TRANSFORM DURATION (ms)
=====
Minimum : 69.1574
Maximum : 71.5955
Average : 70.1912
=====
WRITE-BACK DURATION (ms)
=====
Minimum : 2.8232
Maximum : 760.5902
Average : 6.5888
=====
FPS plot successfully generated and saved to 'perf_cont_sharp_dump_both/fps_performance.png'
Processing time plot successfully generated and saved to 'perf_cont_sharp_dump_both/step_performance.png'
```

Figure 5.8 cont_sharpen frame rate and execution time metrics – DUMP_FRAMES – WRITE_BOTH

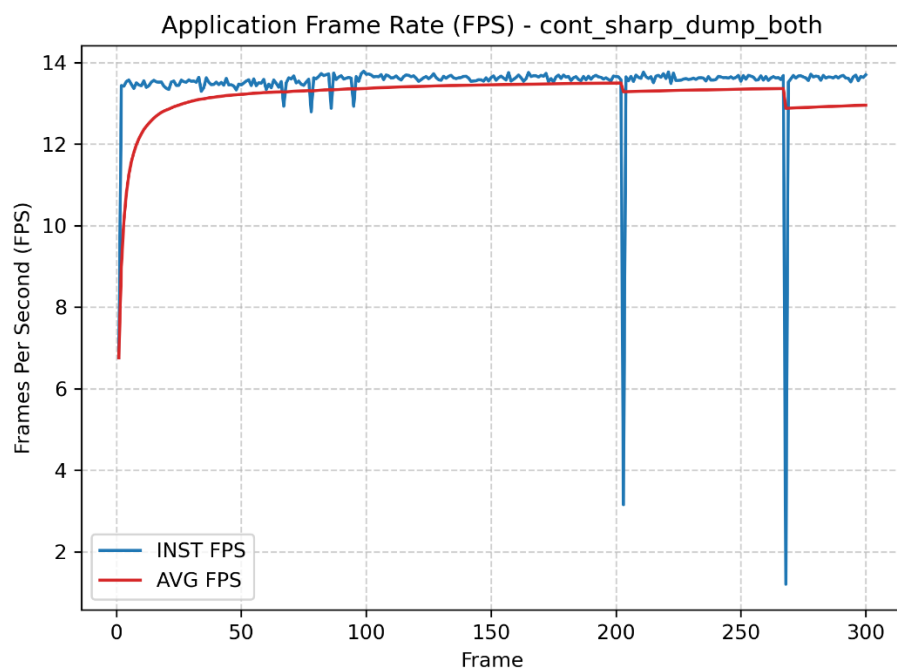


Figure 5.9 cont_sharpen frame rate chart – DUMP_FRAMES – WRITE_BOTH

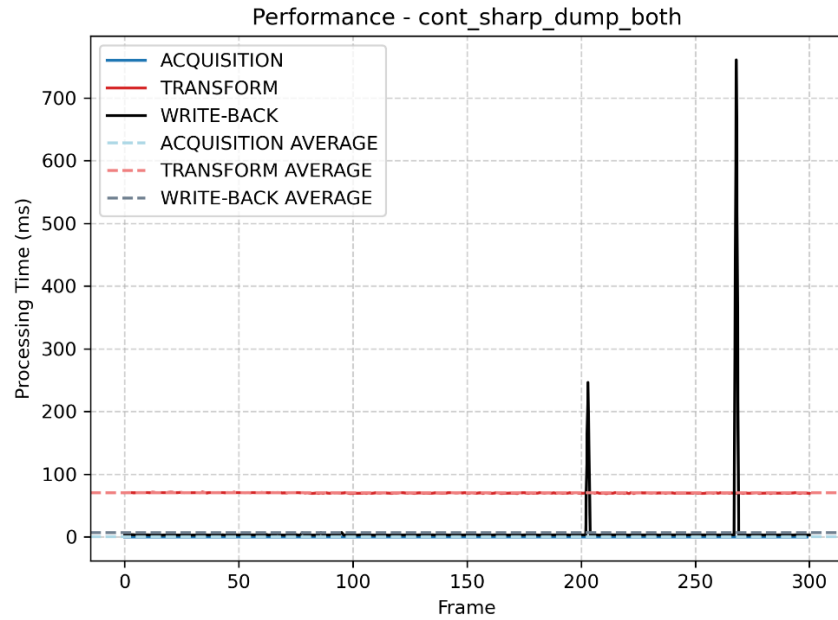


Figure 5.10 cont_sharpen execution time chart – DUMP_FRAMES – WRITE_BOTH

With WRITE_BOTH enabled in DUMP_FRAMES mode, the write-back time per frame approximately doubles relative to the single-output case, because two full 640x480x3-byte PPM files must be written to flash storage per frame cycle. Since write-back was already the dominant bottleneck in part a, this additional burden reduces the average FPS substantially. The acquisition and transform stage timings are unaffected, confirming that the performance degradation is due entirely to I/O.

FFmpeg PIPE

```
mason@jetson-orin-nano:~/RTES_5623/ex4/q5 (RTES_5623::ex4)
$ python3 plot_perf.py

Successfully parsed 300 frames.

=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 6.5859
Maximum FPS : 13.9271
Average FPS : 13.5018
=====
ACQUISITION DURATION (ms)
=====
Minimum : 0.0083
Maximum : 0.0233
Average : 0.0109
=====
TRANSFORM DURATION (ms)
=====
Minimum : 69.0552
Maximum : 75.1734
Average : 70.4763
=====
WRITE-BACK DURATION (ms)
=====
Minimum : 2.3487
Maximum : 13.0658
Average : 3.0950
=====
FPS plot successfully generated and saved to 'perf_cont_sharp_pipe_both/fps_performance.png'
Processing time plot successfully generated and saved to 'perf_cont_sharp_pipe_both/step_performance.png'
```

Figure 5.11 cont_sharpen frame rate and execution time metrics – FFmpeg pipe –
WRITE_BOTH

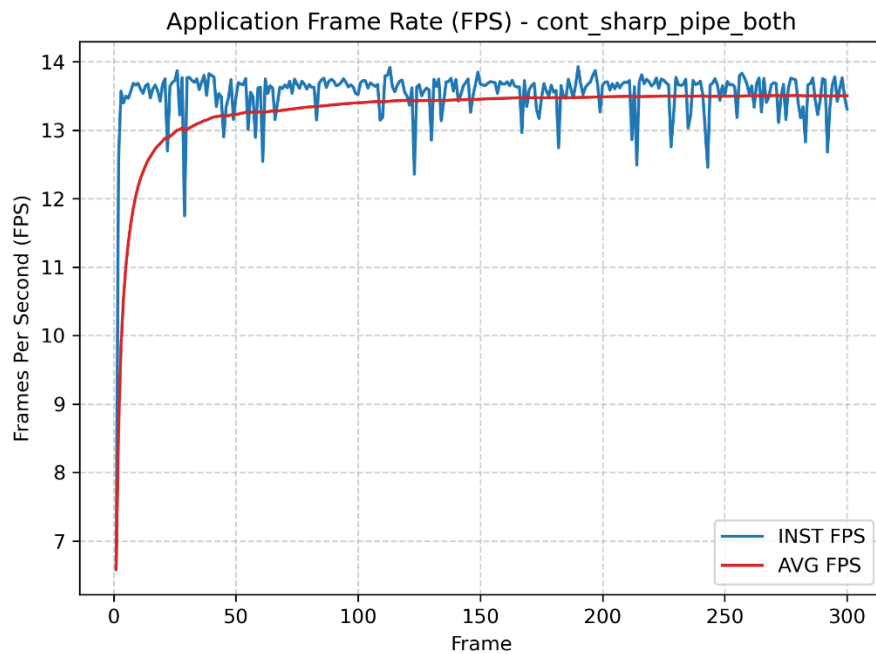


Figure 5.12 cont_sharpen frame rate chart – FFmpeg pipe – WRITE_BOTH

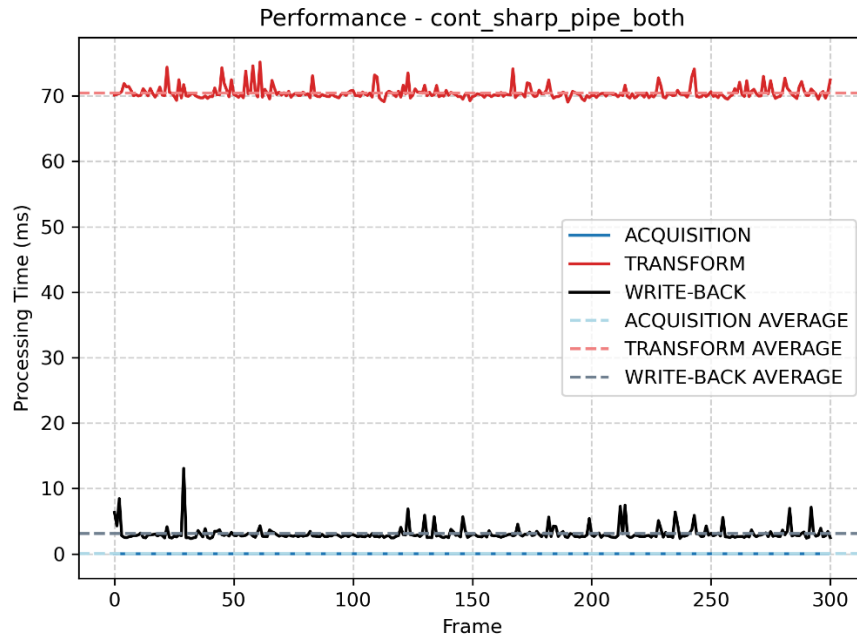


Figure 5.13 cont_sharpen execution time chart – FFmpeg pipe – WRITE_BOTH

With WRITE_BOTH enabled in pipe mode, two sequential calls are made per frame to write to separate FFmpeg pipes. Unlike DUMP_FRAMES mode where each write involves filesystem syscalls, pipe writes depend on FFmpeg buffer availability and encoding throughput. If both FFmpeg processes are running near capacity, the second call may block for longer than the first, adding variable latency. Compared to pipe mode in part a, the average FPS decreases proportionally to the additional pipe write overhead. Compared to WRITE_BOTH w/ DUMP_FRAMES mode, write-back latency in pipe mode is generally lower and more predictable, so the FPS impact of WRITE_BOTH is typically less severe in pipe mode than in dump-frames mode.

c) Sequencer

The seq_sharpen.c application restructures the single-threaded continuous loop from cont_sharpen.c into a two-thread SCHED_FIFO architecture. Period and deadline: DEADLINE_MS is set to 50 milliseconds, corresponding to a 20 Hz target frame rate. A 43 ms period was considered based on average transform time observed in part a but rejected because it left zero margin for acquisition overhead, OS scheduling latency, and write-back variability; any single overrun at 43 ms would cause an immediate missed deadline, whereas 50 ms provides a more realistic soft real-time constraint with headroom for jitter.

The application architecture contains two POSIX threads. The sequencer thread ticks at the 50 ms period using clock_nanosleep() with TIMER_ABSTIME and a monotonically advancing absolute release time, which prevents drift accumulation from relative sleeps. At each tick it increments shared frame_count and calls sem_post() to release the processing thread. The

processing thread blocks on `sem_wait()` and, upon release, dequeues a V4L2 frame, runs the full acquire - transform - write-back pipeline, and logs execution time. If a deadline is missed and the sequencer fires again before the processing thread completes, `sem_post()` increments the semaphore count, causing the next `sem_wait()` to return immediately and triggering back-to-back processing with no idle gap, which is the expected soft real-time missed-deadline behavior.

Scheduling policy and CPU affinity: Both threads are configured with `SCHED_FIFO`. The sequencer runs at the max `SCHED_FIFO` priority minus 2 and the processing thread at max minus 3, ensuring the sequencer can preempt the processing thread to fire ticks on time. Both threads are pinned to CPU Core 1, isolating the real-time workload from general system activity on other cores and making execution times more deterministic.

The `plot_jitter_drift.py` script was adapted from the `plot_perf.py` script by retaining previous functionality and adding one additional field to the original FPS to parse processing execution time (ET). This metric is used to calculate the min/max/average execution time as well as jitter and drift. The script adds the functionality of creating two additional plots – one showing period jitter and the other showing accumulated drift. These plots are organized in the same directory as the two original plots.

Dump Frames

```
mason@jetson-orin-nano:~/RTES_5623/ex4/q5 (RTES_5623:ex4)
$ python3 plot_jitter_drift.py

Successfully parsed 299 frames.
=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 18.3050
Maximum FPS : 21.8315
Average FPS : 19.9449
=====
ACQUISITION DURATION (ms)
=====
Minimum : 0.0080
Maximum : 0.0255
Average : 0.0146
=====
TRANSFORM DURATION (ms)
=====
Minimum : 39.3006
Maximum : 40.1008
Average : 39.5019
=====
WRITE-BACK DURATION (ms)
=====
Minimum : 1.4013
Maximum : 5.7507
Average : 1.6558
=====
EXECUTION TIMES (ms)
=====
Minimum : 40.8346
Maximum : 45.2126
Average : 41.2709
=====
JITTER & DRIFT
=====
Max Jitter : 4.6300
Average Jitter : 0.3805
Max Drift : 4.1970
Average Drift : 0.2365
=====
FPS plot successfully generated and saved to 'perf_seq_sharp_dump/fps_performance.png'
Processing time plot successfully generated and saved to 'perf_seq_sharp_dump/step_performance.png'
Jitter plot successfully saved to 'perf_seq_sharp_dump/jitter.png'
Drift plot successfully saved to 'perf_seq_sharp_dump/drift.png'
```

Figure 5.14 seq_sharpen frame rate and execution time metrics – DUMP_FRAMES

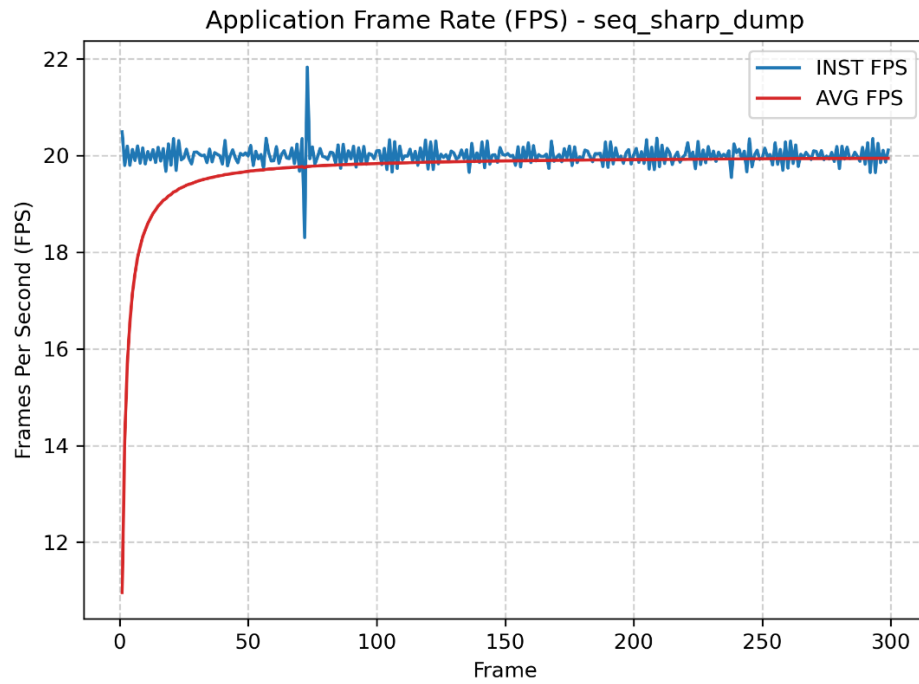


Figure 5.15 seq_sharpen frame rate chart – DUMP_FRAMES

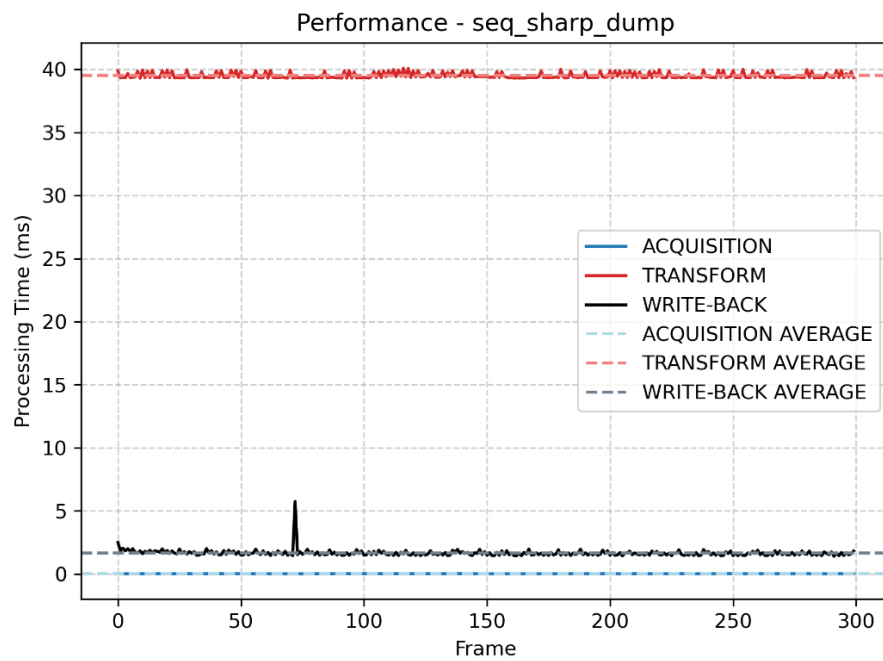


Figure 5.16 seq_sharpen execution time chart – DUMP_FRAMES

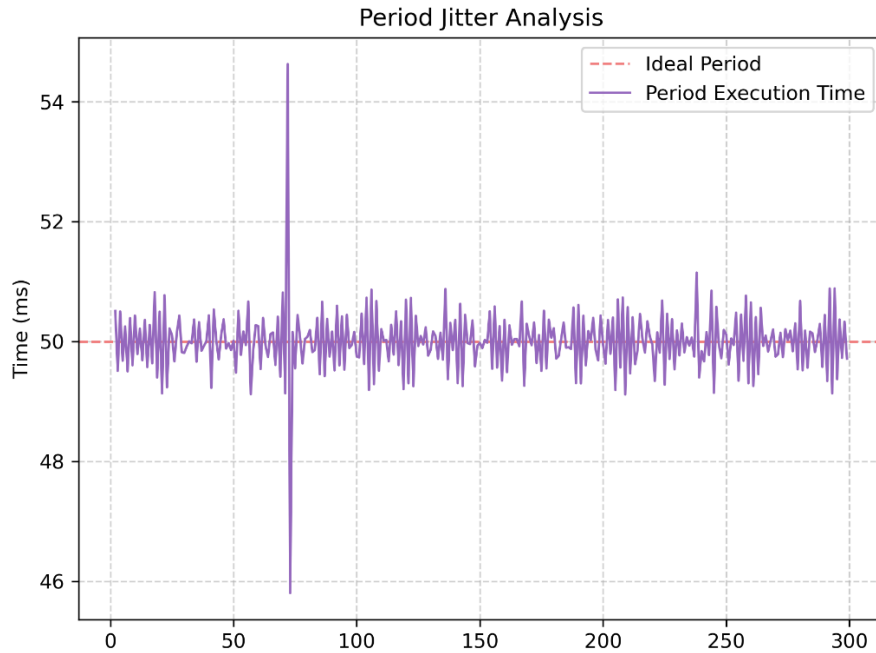


Figure 5.17 seq_sharpen period jitter analysis – DUMP_FRAMES

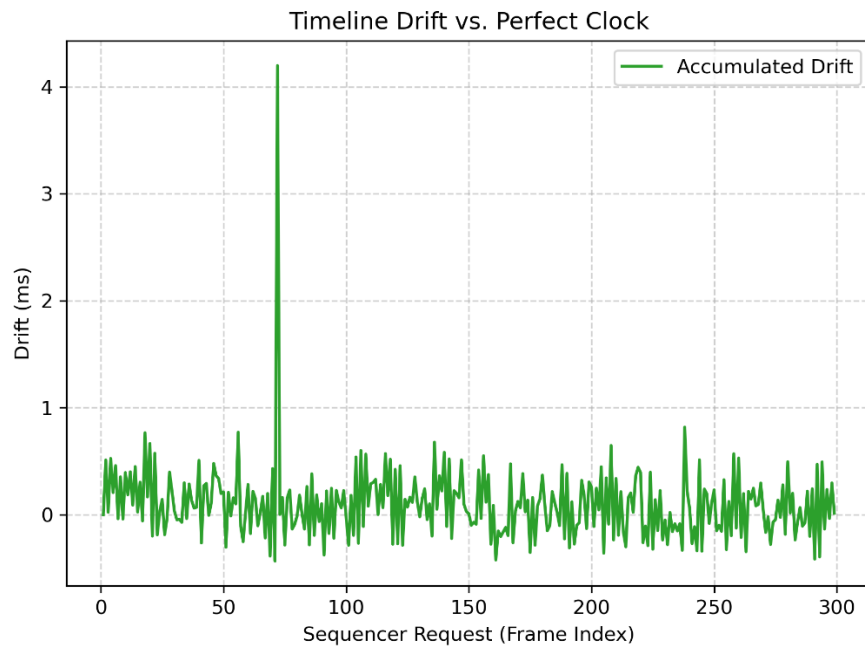


Figure 5.18 seq_sharpen accumulated drift – DUMP_FRAMES

In dump-frames mode with the sequencer, the period jitter plot shows the measured inter-frame period against the ideal 50 ms reference. Under SCHED_FIFO with the sequencer thread at maximum priority, the tick interval is tightly controlled, with jitter primarily introduced by `clock_nanosleep()` overshoot and kernel scheduling granularity. The execution time logged per

frame captures the total time from `sem_wait()` release through acquisition, transform, and write-back.

In dump-frames mode, write-back to flash storage is typically the dominant contributor to ET variability. Flash I/O latency spikes can push individual frames past the 50 ms budget, producing visible period excursions in the jitter plot. The drift plot tracks cumulative deviation from a perfect 50 ms clock; correct absolute-time ticking should keep drift bounded and oscillating around zero rather than growing monotonically. Any monotonically increasing drift indicates a sustained overload condition where the workload consistently exceeds 50 ms. The average FPS should converge near 20 Hz in nominal cycles, with occasional dips corresponding to flash write spikes that cause missed deadlines.

FFmpeg PIPE

```
^Cmason@jetson-orin-nano:~/RTES_5623/ex4/q5 (RTES_5623::ex4)
$ python3 plot_jitter_drift.py

Successfully parsed 299 frames.

=====
FPS PERFORMANCE METRICS
=====
Minimum FPS : 19.0685
Maximum FPS : 21.4082
Average FPS : 19.9446
=====

ACQUISITION DURATION (ms)
=====
Minimum : 0.0141
Maximum : 1.0114
Average : 0.0324
=====

TRANSFORM DURATION (ms)
=====
Minimum : 39.2025
Maximum : 42.9464
Average : 39.6136
=====

WRITE-BACK DURATION (ms)
=====
Minimum : 0.6822
Maximum : 4.3584
Average : 1.5430
=====

EXECUTION TIMES (ms)
=====
Minimum : 40.0544
Maximum : 44.6185
Average : 41.3892
=====

JITTER & DRIFT
=====
Max Jitter : 3.2890
Average Jitter : 0.6558
Max Drift : 2.4310
Average Drift : 0.8865
=====

FPS plot successfully generated and saved to 'perf_seq_sharp_pipe/fps_performance.png'
Processing time plot successfully generated and saved to 'perf_seq_sharp_pipe/step_performance.png'
Jitter plot successfully saved to 'perf_seq_sharp_pipe/jitter.png'
Drift plot successfully saved to 'perf_seq_sharp_pipe/drift.png'
```

Figure 5.19 seq_sharpen frame rate and execution time metrics – FFmpeg pipe

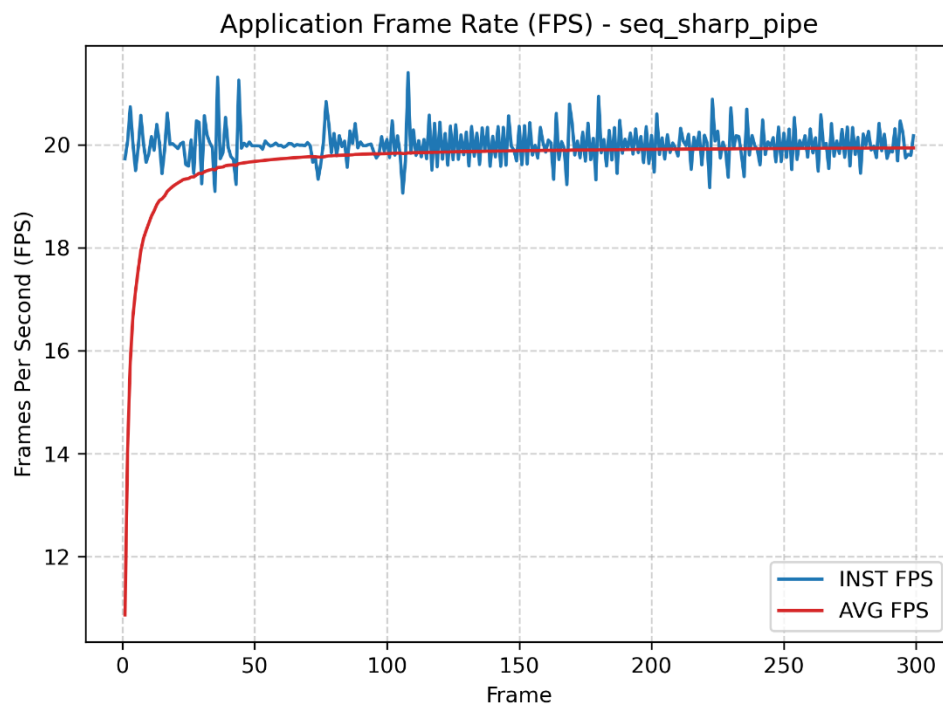


Figure 5.20 seq_sharpen frame rate chart – FFmpeg pipe

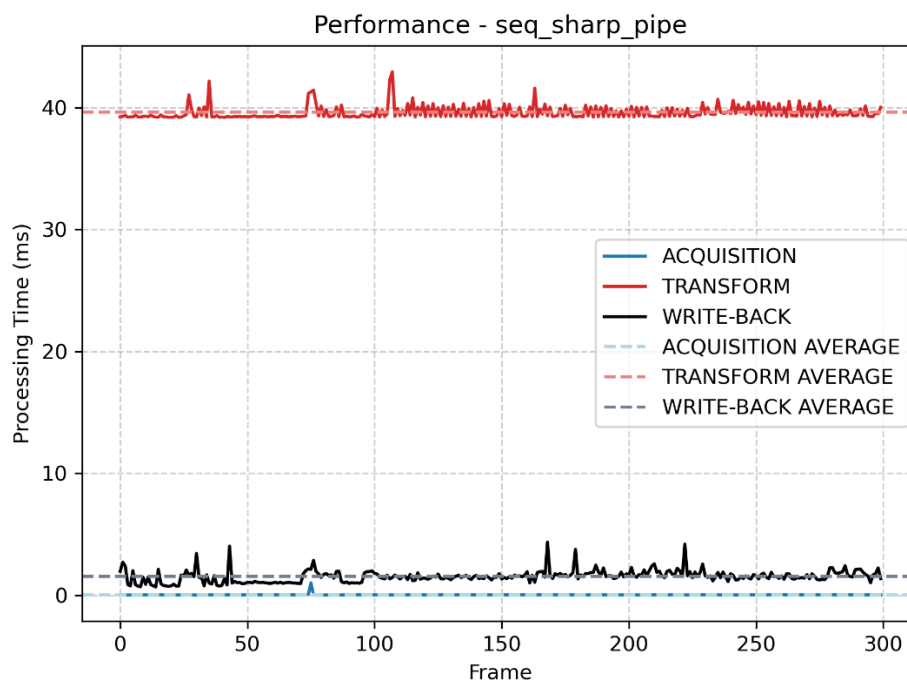


Figure 5.21 seq_sharpen execution time chart – FFmpeg pipe

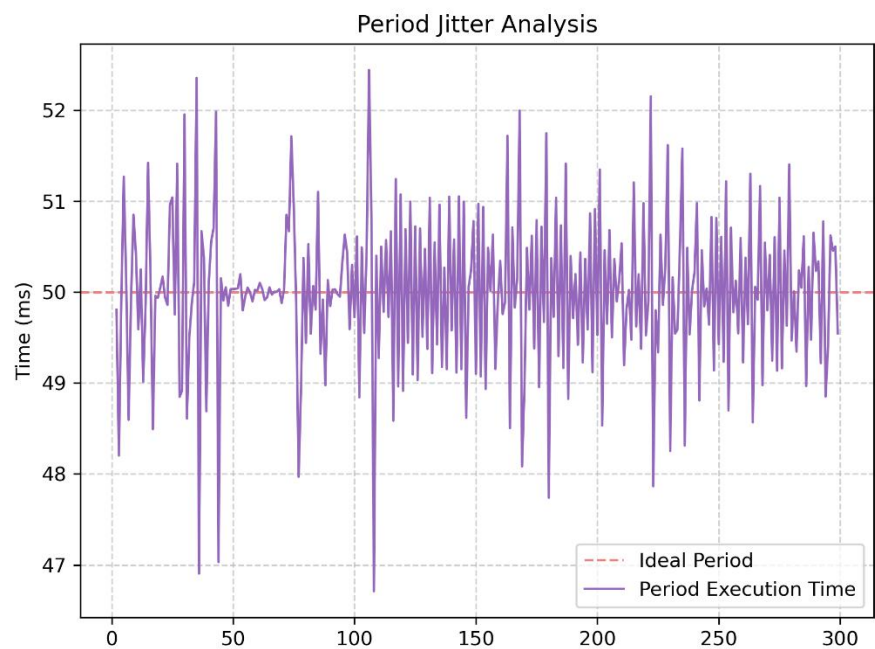


Figure 5.22 seq_sharpen period jitter analysis – FFmpeg pipe

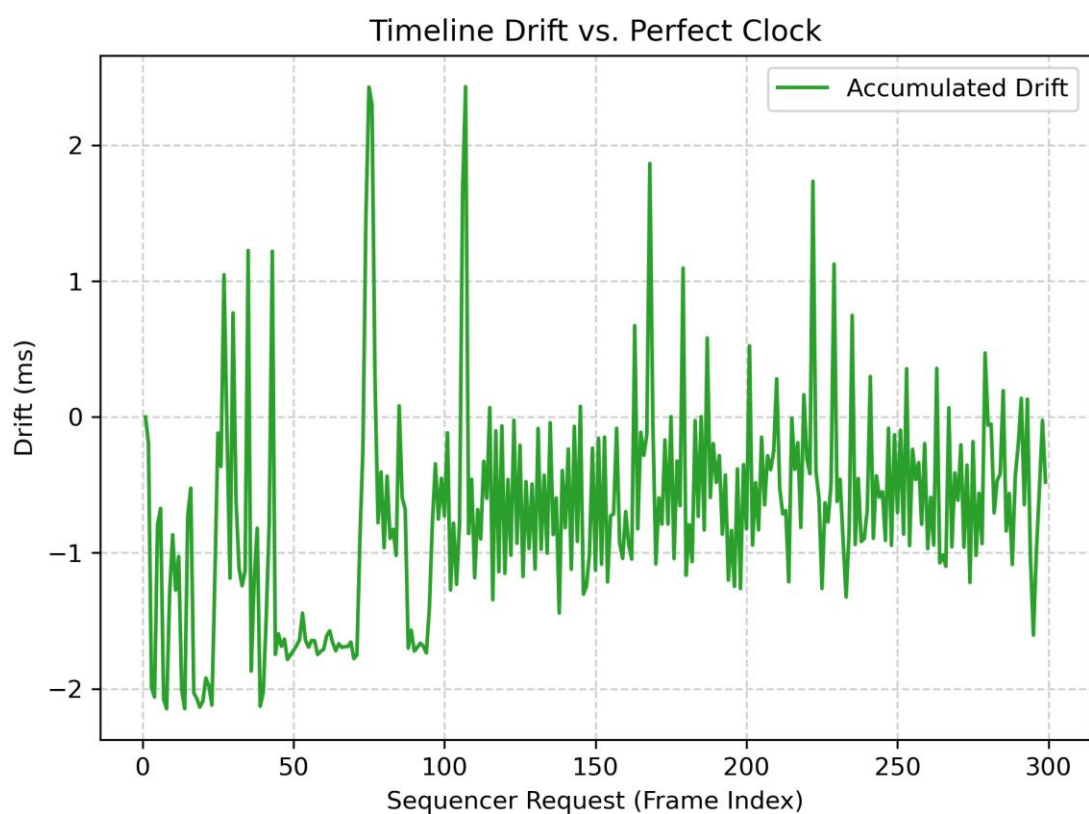


Figure 5.23 seq_sharpen accumulated drift – FFmpeg pipe

In pipe mode with the sequencer, write-back goes into a FFmpeg pipe rather than individual flash file writes, which generally reduces and smooths write-back latency. As a result, per-frame ET values are more consistent compared to dump-frames mode, meaning fewer deadline misses and tighter period jitter. The jitter plot should show a narrower spread around the 50 ms ideal period, and the drift plot should demonstrate better long-term stability. The average FPS in pipe mode should be closer to the 20 Hz target than in dump-frames mode, reflecting the more predictable write-back path.

Comparing the two output modes directly, the primary difference in sequencer behavior is the write-back contribution to ET. In pipe mode, FFmpeg encoding throughput becomes the limiting factor rather than flash storage latency. If FFmpeg processes frames faster than they arrive at 20 Hz, the pipe buffer drains freely, keeping ET well within the 50 ms budget. If FFmpeg falls behind, pipe backpressure causes it to block, reintroducing latency. The sequencer absolute-time ticking suppresses accumulated drift in both modes as long as the workload does not consistently exceed the 50 ms period.

6 References

- [1] J. Pratt and S. Siewert, Real-Time Embedded Components and Systems with Linux and RTOS, 2nd ed. Herndon, VA, USA: Mercury Learning and Information, 2016.
- [2] "Linux man pages," Die.net. Accessed: Jun. 5, 2026. [Online]. Available: <https://linux.die.net/man/>
- [3] M. Kerrisk, "Linux man-pages online," Man7.org. Accessed: Jun. 5, 2026. [Online]. Available: <http://man7.org/linux/man-pages/index.html>
- [4] S. Siewert, "SoC drawer: Shared resource management," *Real-Time Embedded Systems (RTES) Course Materials*, University of Colorado Boulder. [Online].
- [5] BatchLoaf, "A simple way to read and write audio and video files in C++ using FFmpeg — Part 2: Video," *BatchLoaf*, Feb. 12, 2017. [Online]. Available: <https://batchloaf.wordpress.com/2017/02/12/a-simple-way-to-read-and-write-audio-and-video-files-in-c-using-ffmpeg-part-2-video/>
- [6] "How can I setup FFmpeg to continuously produce 1 second videos from piped jpgs," *Super User*, Nov. 22, 2023. [Online]. Available: <https://superuser.com/questions/1812357/how-can-i-setup-ffmpeg-to-continuously-produce-1-second-videos-from-piped-jpgs>
- [7] "Piping ppm files to FFmpeg to create movie in C," *Stack Overflow*, Aug. 14, 2015. [Online]. Available: <https://stackoverflow.com/questions/32024859/piping-ppm-files-to-ffmpeg-to-create-movie-in-c>

7 Appendix

GitHub repository: https://github.com/mamc3334/RTES_5623

Pull request: https://github.com/mamc3334/RTES_5623/pull/4

Forked RTES GitHub repository: <https://github.com/mamc3334/RTES-ECEE-5623-mcgaffin>

Pull request: <https://github.com/mamc3334/RTES-ECEE-5623-mcgaffin/pull/4>